

Sprawozdanie

**Projekt Algorytmy i Złożoność
Obliczeniowa**

Wykonawca

Hanna Selerowicz

280715

Spis treści

Wstęp	2
Cel projektu	2
Założenia projektu.....	2
Algorytmy i ich implementacja	3
1. Przez wstawianie	3
Zwykłe (Insertion Sort)	3
Binarne (Binary Insertion Sort)	4
2. Sortowanie szybkie (Quick Sort)	5
3. Sortowanie przez kopcowanie (Heap Sort).....	6
4. Wersja pijanego studenta (Drunk insertion sort)	7
Badania	9
Wpływ liczby elementów.....	9
1. Sortowanie przez wstawianie	9
2. Sortowanie szybkie	14
3. Sortowanie przez kopcowanie.....	15
Wpływ początkowego rozkładu danych	17
1. Sortowanie przez wstawianie	18
2. Sortowanie szybkie	22
3. Sortowanie przez kopcowani	24
Wpływ typów danych	27
Wersja pijanego studenta.....	29
1. Wpływ stopnia pijaństwa	29
2. Wpływ początkowego rozkładu danych.....	31
3. Wpływ typów danych	33
Podsumowanie	36

Wstęp

Cel projektu

Celem projektu było zaimplementowanie i przetestowanie kilku popularnych algorytmów sortowania oraz ich modyfikacji, w tym zarówno klasycznych metod, jak i wersji z losowym zakłóceniem – „pijany student”. Główna idea polegała na porównaniu wydajności różnych algorytmów sortujących w kontekście ich czasu wykonania, a także na zrozumieniu, jak różne techniki (np. losowość, zmiana porządku sortowania) mogą wpływać na efektywność sortowania w praktycznych zastosowaniach.

Założenia projektu

1. **Wybór algorytmów:** Projekt obejmował implementację i testowanie kilku algorytmów sortowania, w tym:
 - Sortowanie przez wstawianie (klasyczne i binarne).
 - Sortowanie szybkie.
 - Sortowanie przez kopcowanie.
 - Modyfikacja klasycznego sortowania przez wstawianie w wersji „pijany student” z losowym pomijaniem liczb.
2. **Testy porównawcze:** Każdy algorytm został przetestowany 100-krotnie, przy użyciu nowego, losowo generowanego zbioru danych w każdej próbie, co miało zapewnić wiarygodność i powtarzalność wyników. Pomiary wykonano w różnych warunkach, uwzględniając zróżnicowane cechy danych wejściowych. W badaniach uwzględniono kilka kluczowych aspektów:
 - **Rozmiar danych:** Testy przeprowadzono na tablicach o rozmiarach 10, 20, 30, 40 i 50 tysięcy elementów typu int, aby ocenić skalowalność algorytmów i ich wydajność przy rosnącej liczbie danych.
 - **Układ danych:** Dla tablic o stałym rozmiarze 30 tysięcy elementów analizowano różne układy danych – całkowicie losowy, uporządkowany rosnąco, malejąco oraz częściowo uporządkowany w 33% i 66%. Celem było określenie wpływu wstępnego uporządkowania danych na szybkość działania algorytmu.
 - **Typ danych:** Wykorzystując szybkie sortowanie, zbadano wpływ typu danych (int, float, char) na efektywność przetwarzania przy stałym rozmiarze tablicy (30 tysięcy elementów).
 - **Stopień pijaństwa:** Dla algorytmu pijanego studenta analizowano wpływ poziomu losowego pomijania elementów (10%, 25%, 50%, 75%, 90%) przy tablicy zawierającej 10 tysięcy elementów typu int.
3. **Porównanie wyników:** Zebrane dane umożliwiły przeprowadzenie szczegółowej analizy porównawczej wszystkich algorytmów w zróżnicowanych warunkach. Każdy algorytm oceniano osobno, analizując zmiany czasu wykonania w zależności od typu

danych, ich układu, rozmiaru oraz specyficznych modyfikacji (np. poziomu "pijaństwa"). Wyniki przedstawiono w formie tabelarycznej oraz graficznej, co umożliwiło jednoznaczne wskazanie tendencji i zależności.

Algorytmy i ich implementacja

1. Przez wstawianie

Zwykle (Insertion Sort)

Sortowanie przez wstawianie to jeden z najprostszych algorytmów sortowania. Polega na przechodzeniu przez tablicę od drugiego elementu i wstawianiu każdego kolejnego elementu w odpowiednie miejsce w posortowanej części tablicy. Działa to trochę jak wstawianie kart do ręki podczas gry w karty – za każdym razem bierzemy jedną kartę (element) i umieszczamy ją w odpowiedniej pozycji w już posortowanej części zbioru.

Zalety:

- Łatwy do implementacji.
- Bardzo efektywny dla małych zbiorów danych.

Wady:

- Dla dużych danych jest wolniejszy niż inne algorytmy, ponieważ jego złożoność czasowa wynosi $O(n^2)$.

Implementacja oraz działanie (poprawna i nie)

POPRAWNY:

```
template<typename T>
void Sorter<T>::insertionSort(T* array, size_t size) {
    for (int i = 1; i < static_cast<int>(size); ++i) {
        T key = array[i];
        int j = i - 1;
        while (j >= 0 && array[j] > key) {
            array[j + 1] = array[j];
            --j;
        }
        array[j + 1] = key;
    }
}
```

NIEPOPRAWNY*:

```
template<typename T>
void Sorter<T>::insertionSort(T* array, size_t size) {
    for (size_t i = 1; i < size; ++i) {
        T key = array[i];
        size_t j = i - 1;
        while (j >= 0 && array[j] > key) {
            array[j + 1] = array[j];
            --j;
        }
        array[j + 1] = key;
    }
}
```

*W niepoprawnej wersji zmienna `j` była zadeklarowana jako `size_t` – czyli **typ bez znaku**. W momencie, gdy `j` osiągało wartość 0 i było dekrementowane (`--j`), jego wartość nie spadała do -1, tylko „zawijała się” do bardzo dużej liczby (np. 4294967295 przy 32-bitowym `size_t`). To prowadziło do błędów

1. Zaczynamy od drugiego elementu w tablicy, ponieważ pierwszy element traktujemy jako już posortowany.
2. Dla każdego kolejnego elementu porównujemy go z elementami poprzednimi i przesuwamy te, które są większe, o jedno miejsce w prawo, aż znajdziemy odpowiednie miejsce dla bieżącego elementu.
3. Po znalezieniu odpowiedniego miejsca wstawiamy element w tę lukę, a algorytm przechodzi do następnego elementu.
4. Proces powtarza się, aż wszystkie elementy będą posortowane.

Binarne (Binary Insertion Sort)

Sortowanie przez wstawianie binarne to modyfikacja klasycznego sortowania przez wstawianie, polega na tym, że bierzemy każdy element z tablicy po kolei i wkładamy go w odpowiednie miejsce w posortowanej już części. Zamiast sprawdzać to miejsce „po kolei”, szukamy je szybciej — używając wyszukiwania binarnego (czyli dzieląc na pół i porównując), żeby szybciej znaleźć, gdzie go włożyć. Potem przesuwamy większe elementy w prawo i wstawiamy ten jeden element w znalezione miejsce. Działa to podobnie jak układanie kart w rękę, ale z szybkim sprawdzaniem, gdzie pasuje nowa karta. Dzięki temu zmniejszamy liczbę porównań, ponieważ czas wyszukiwania miejsca w posortowanej części tablicy skraca się z $O(n)$ do $O(\log n)$.

Zalety:

- Mniej porównań niż w klasycznym sortowaniu przez wstawianie.
- Nadal łatwy w implementacji.

Wady:

- Nadal jest to algorytm o złożoności $O(n^2)$, ponieważ przesunięcia elementów są nadal wykonywane liniowo.

Implementacja oraz działanie

```
template<typename T>
void Sorter<T>::binaryInsertionSort(T* array, size_t size) {
    if (!array || size == 0) return;

    for (size_t i = 1; i < size; ++i) {
        T key = array[i];
        int left = 0;
        int right = static_cast<int>(i) - 1;

        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (array[mid] > key)
                right = mid - 1;
            else
                left = mid + 1;
        }

        for (int j = static_cast<int>(i); j > left; --j)
```

```

        array[j] = array[j - 1];
        array[left] = key;
    }
}

```

1. Podobnie jak w klasycznym sortowaniu przez wstawianie, zaczynamy od drugiego elementu, traktując pierwszy element jako posortowany.
2. Dla każdego kolejnego elementu, zamiast porównywać go kolejno z wszystkimi wcześniejszymi elementami, używamy wyszukiwania binarnego, aby znaleźć miejsce, w którym element powinien być wstawiony w posortowanej części tablicy.
3. Po znalezieniu miejsca przesuwamy elementy o jedno miejsce w prawo, aby zrobić miejsce dla nowego elementu.
4. Po wstawieniu elementu w odpowiednie miejsce, algorytm przechodzi do kolejnego elementu i powtarza proces, aż tablica zostanie posortowana.

2. Sortowanie szybkie (Quick Sort)

Sortowanie szybkie to jeden z najszybszych algorytmów sortowania w praktyce. Opiera się na zasadzie „dziel i zwyciężaj” – wybieramy element, zwany pivot (w naszym przypadku jest to środkowy element), i dzielimy tablicę na dwie części: jedną z elementami mniejszymi od piwota i drugą z elementami większymi. Następnie rekurencyjnie powtarzamy ten proces dla obu części.

Zalety:

- Bardzo szybki w praktyce.
- Złożoność średnia to $O(n \log n)$, chociaż w najgorszym przypadku może wynosić $O(n^2)$.

Wady:

- W najgorszym przypadku (np. dla już posortowanej tablicy) może być bardzo wolny. W praktyce unika się tego, stosując różne techniki.

Implementacja oraz działanie

```

template<typename T>
void Sorter<T>::quickSort(T* array, int left, int right) {
    if (left >= right) return;
    T pivot = array[(left + right) / 2];
    int i = left, j = right;
    while (i <= j) {
        while (array[i] < pivot) ++i;
        while (array[j] > pivot) --j;
        if (i <= j) std::swap(array[i++], array[j--]);
    }
    if (left < j) quickSort(array, left, j);
    if (i < right) quickSort(array, i, right);
}

```

1. Algorytm zaczyna od wyboru elementu *pivot* — w tym przypadku jest to element znajdujący się w środku tablicy, na pozycji $(\text{left} + \text{right}) / 2$.
2. Następnie tablica jest reorganizowana w taki sposób, że elementy mniejsze od *pivota* przesuwają się na jego lewą stronę, a większe na prawą. Dzieje się to przez przesuwanie wskaźników i (początek tablicy) i j (koniec tablicy), które porównują wartości z *pivota* i przesuwają elementy w odpowiednie miejsca.
3. Gdy oba wskaźniki przekroczą się, to tablica jest podzielona na dwie części: lewą (mniejsze od *pivota*) i prawą (większe od *pivota*).
4. Algorytm rekurencyjnie sortuje te dwie części. Proces dzielenia i sortowania powtarza się, aż tablica zostanie posortowana.

3. Sortowanie przez kopcowanie (Heap Sort)

Sortowanie przez kopcowanie to algorytm oparty na strukturze danych zwanej kopcem (heap). Działa on na zasadzie budowania kopca z tablicy, a następnie wielokrotnego usuwania największego elementu (w przypadku kopca maksymalnego) i umieszczania go na końcu tablicy. Po każdej operacji usunięcia elementu kopiec jest odbudowywany, aby zachować swoje właściwości.

Zalety:

- Złożoność czasowa $O(n \log n)$ w każdym przypadku (zarówno średnim, jak i w najgorszym).
- Nie wymaga dodatkowej pamięci (sortowanie w miejscu).

Wady:

- Jest wolniejszy od innych algorytmów o złożoności $O(n \log n)$, takich jak sortowanie szybkie, w praktyce, ponieważ wymaga wielu operacji na strukturach danych (kopcach).

Implementacja oraz działanie

```
template<typename T>
void Sorter<T>::heapify(T* array, size_t size, size_t root) {
    size_t largest = root;
    size_t left = 2 * root + 1;
    size_t right = 2 * root + 2;

    if (left < size && array[left] > array[largest])
        largest = left;
    if (right < size && array[right] > array[largest])
        largest = right;

    if (largest != root) {
        std::swap(array[root], array[largest]);
        heapify(array, size, largest);
    }
}
```

```
template<typename T>
void Sorter<T>::heapSort(T* array, size_t size) {
    for (size_t i = size / 2; i-- > 0; )
        heapify(array, size, i);
    for (size_t i = size - 1; i > 0; --i) {
        std::swap(array[0], array[i]);
        heapify(array, i, 0);
    }
}
```

1. Budowanie kopca (heapify):

- Algorytm zaczyna od przekształcenia tablicy w kopiec maksymalny. Oznacza to, że dla każdego węzła (elementu) tablicy zachowywana jest właściwość kopca, czyli elementy rodzica są większe lub równe elementom dzieci.
- Zaczynamy od połowy tablicy (najdalsze węzły liściowe) i przechodzimy do korzenia. Dla każdego elementu na poziomie rodziców wykonujemy operację *heapify*, która zapewnia, że poddrzewo o korzeniu w danym węźle spełnia warunki kopca.

2. Sortowanie:

- Po zbudowaniu kopca, elementy znajdujące się na początku tablicy (czyli korzeń kopca) są największe. Dlatego przestawiamy pierwszy element tablicy (korzeń kopca) na ostatnią pozycję, zmniejszamy rozmiar kopca o 1 i ponownie wykonujemy operację *heapify* na nowym korzeniu, aby przywrócić właściwości kopca.
- Proces ten powtarzamy, aż wszystkie elementy znajdą się na odpowiednich miejscach w posortowanej tablicy.

3. Operacja heapify:

- Funkcja *heapify* jest kluczowym elementem tego algorytmu. Jej zadaniem jest zapewnienie, że w poddrzewie o korzeniu w danym węźle, wartości rodzica są większe lub równe wartościom jego dzieci. W przypadku naruszenia tej zasady, funkcja wykonuje rekurencyjne zamiany, aż poddrzewo będzie spełniać wymagania kopca.

4. Wersja pijanego studenta (Drunk insertion sort)

Pijany student to ciekawa modyfikacja klasycznego algorytmu sortowania przez wstawianie. W tym przypadku student (algorytm) ma określony % szans na to, że pomiędzy przesunięciami pominie jakiś element i nie będzie go sortował w danym przejściu. Dzięki temu algorytm staje się mniej przewidywalny, ale ciągle wykonuje kolejne próby (nie poddaje się), aż tablica zostanie posortowana. Funkcja działa do momentu, w którym cała tablica nie jest uporządkowana, wykonując ponowne próby w razie potrzeby.

Zalety:

- Interesujący sposób na „zaburzenie” klasycznego algorytmu.
- Przykład użycia losowości w algorytmach sortowania.

Wady:

- W momencie kiedy stopień pijaństwa wynosi 100% student "zaśnie" i nie wykona algorytmu
- W większości przypadków nie jest efektywny (lepiej wykonywać na trzeźwo)

Implementacja oraz działanie

```
template<typename T>
void Sorter<T>::drunkInsertionSort(T* array, size_t size) {
    std::mt19937 rng(std::random_device{}());
    std::uniform_int_distribution<int> chance(1, 100);
    int drunkness = 75; // % szans, że student pominie liczbę

    while (!isSorted(array, size)) {
        for (size_t i = 1; i < size; ++i) {
            // Student może pominąć ten element
            if (chance(rng) <= drunkness) {
                continue; // nie chce mu się sortować tej liczby
            }

            T key = array[i];
            int j = static_cast<int>(i) - 1;
            while (j >= 0 && array[j] > key) {
                array[j + 1] = array[j];
                --j;
            }
            array[j + 1] = key;
        }
    }
}
```

1. **Warunki początkowe:** Algorytm zaczyna działać od tablicy, której elementy mogą być w dowolnym porządku. Używa generatora liczb losowych (`std::mt19937` i `std::uniform_int_distribution`), aby określić, z jakim prawdopodobieństwem elementy będą pomijane.
2. **Losowe pomijanie elementów:** W pętli głównej (gdzie algorytm sortuje tablicę) dla każdego elementu istnieje losowe prawdopodobieństwo, że element ten zostanie "pominięty" przez algorytm, czyli nie zostanie uwzględniony w sortowaniu. Jeśli liczba losowa jest mniejsza lub równa zadanej wartości *drunkness* (np. 75%), algorytm przechodzi do następnego elementu, ignorując aktualny. W praktyce oznacza to, że student może zignorować pewne liczby i nie próbować ich posortować.
3. **Sortowanie przez wstawianie:** Dla elementów, które nie zostały pominięte, algorytm wykonuje tradycyjne sortowanie przez wstawianie. Przemieszcza każdy element na odpowiednią pozycję w posortowanej części tablicy, porównując go z poprzednimi elementami i przesuwając je w prawo, dopóki nie znajdzie odpowiedniego miejsca na wstawienie.
4. **Powtarzanie aż do posortowania:** Cały proces powtarza się, aż cała tablica będzie posortowana. Pętla główna działa, dopóki tablica nie jest całkowicie posortowana, czyli dopóki wynik funkcji sprawdzającej (`isSorted`) nie zwróci wartości prawdziwej. Jest to konieczne, ponieważ pomijanie elementów może sprawić, że algorytm nie będzie

działał zgodnie z oczekiwaniami za pierwszym razem, więc musi on "próbować ponownie", aż wszystkie elementy zostaną uwzględnione i posortowane.

Badania

Wpływ liczby elementów

W celu zbadania wydajności algorytmów sortowania, przeprowadzono pomiary czasu ich działania dla tablic zawierających od 10 000 do 50 000 elementów typu int. Wyniki przedstawiono w postaci tabeli statystyk zawierającej czasy minimalne, maksymalne, średnie oraz mediany (wszystkie wartości w milisekundach).

1. Sortowanie przez wstawianie

Zwykle

Błędne:

Liczba elementów	10000	20000	30000	40000	50000
Min [ms]	63	255	575	1027	1603
Max [ms]	78	278	612	1112	2162
Średnia [ms]	64,96	259,59	585,32	1046,15	1637,46
Mediana [ms]	64	259	584	1040	1620

Tabela 1.1. Błędny czas działania algorytmu Insertion Sort w zależności od liczby elementów

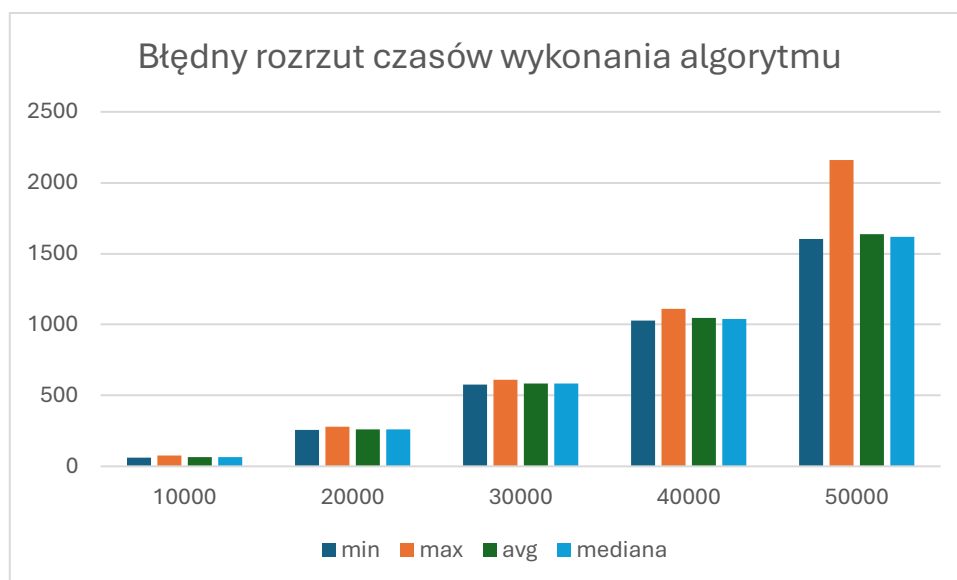
Poprawione:

Liczba elementów	10000	20000	30000	40000	50000
Min [ms]	49	201	462	869	1377
Max [ms]	113	356	949	1245	2315
Średnia [ms]	57,65	213,17	528,12	959,02	1591,13
Mediana [ms]	52	207	511	958	1554

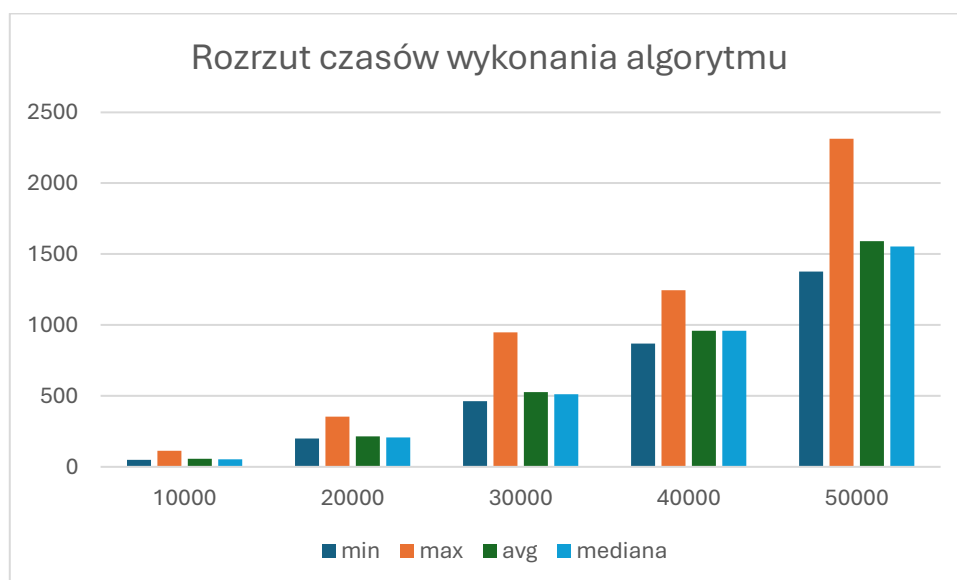
Tabela 1.2. Czas działania algorytmu Insertion Sort w zależności od liczby elementów

Wzrost czasu sortowania

Z wykresu kolumnowego oraz danych zawartych w tabeli widać wyraźnie, że czas sortowania rośnie wraz ze zwiększaniem się rozmiaru danych wejściowych. Przykładowo, dla 10 000 elementów średni czas sortowania wynosił ok. 58 ms, natomiast dla 50 000 elementów był to już czas rzędu 1591,13 ms. Pokazuje to, że czas wykonania wzrasta szybciej niż proporcjonalnie do liczby danych, co jest zgodne z teoretyczną złożonością algorytmu — $O(n^2)$.



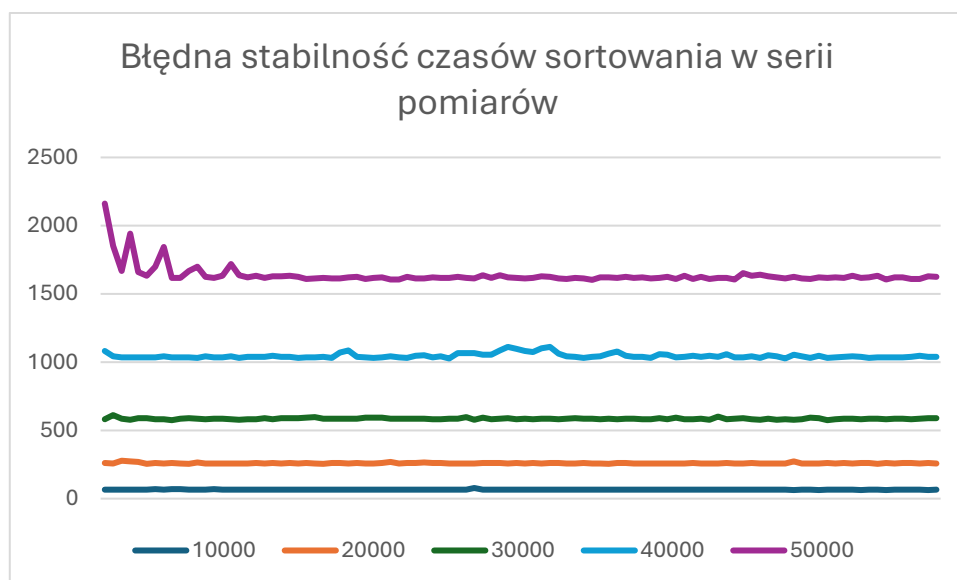
Wykres 1.1. Błędny rozrzut czasów wykonania algorytmu Insertion Sort w zależności od ilości danych



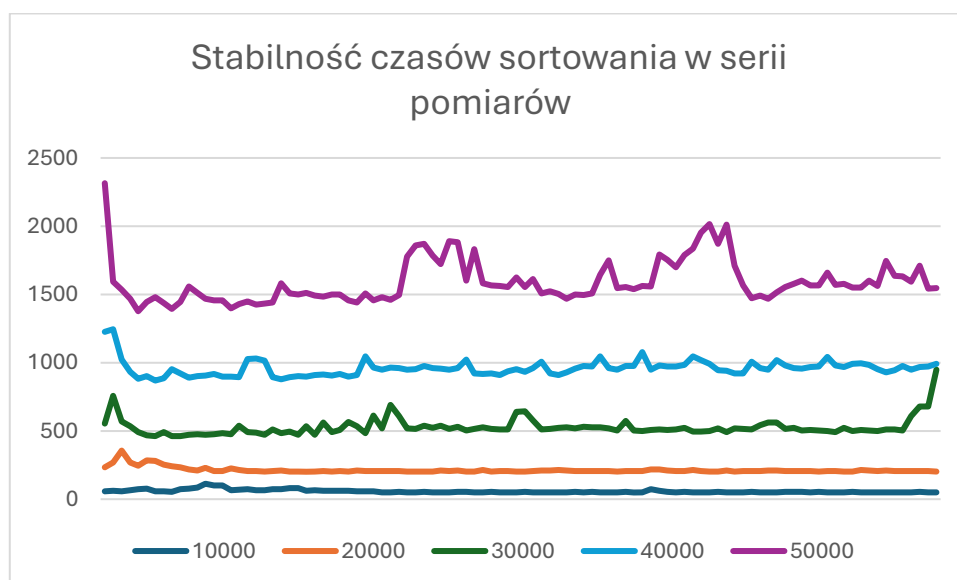
Wykres 1.2. Rozrzut czasów wykonania algorytmu Insertion Sort w zależności od ilości danych

Stabilność pomiarów

Zarówno wykres liniowy przedstawiający poszczególne pomiary, jak i dane w tabeli wskazują na wyraźnie większą rozpiętość między wartościami minimalnymi a maksymalnymi dla każdego rozmiaru tablicy po poprawie implementacji. Przykładowo, dla 30 000 elementów czasy wahają się w zakresie od 462 do 949 ms, co daje rozrzut aż 487 ms. Oznacza to, że algorytm nie zachowuje się już tak stabilnie czasowo, jak wcześniej, i jest wrażliwy na konkretne ułożenie danych wejściowych przy danej liczbie elementów. Takie zachowanie jest typowe dla algorytmu sortowania przez wstawianie, którego wydajność może się znacząco różnić w zależności od stopnia posortowania danych. Wcześniejsza wersja wykazywała nienaturalnie małą rozpiętość czasów z powodu błędu implementacyjnego.



Wykres 2.1. Błędna stabilność czasów sortowania w serii pomiarów algorytmu Insertion Sort w zależności od ilości danych



Wykres 2.2. Stabilność czasów sortowania w serii pomiarów algorytmu Insertion Sort w zależności od ilości danych

Wydajność dla małych i dużych zbiorów

Dla mniejszych tablic (10 000–20 000 elementów) algorytm działa bardzo szybko, co czyni go przydatnym w prostych zastosowaniach lub przy niewielkiej skali danych. Jednak dla większych rozmiarów (40 000–50 000) czasy wykonania przekraczają 1 sekundę, co sprawia, że algorytm staje się nieefektywny.

Wnioski

- Wydajność algorytmu spada szybko wraz ze wzrostem liczby danych, co wynika z jego kwadratowej złożoności.

- Po poprawce implementacji czasy wykonania wykazują większą zmienność przy kolejnych uruchomieniach, co jest naturalne dla algorytmu o złożoności $O(n^2)$ działającego na danych losowych.
- Insertion Sort sprawdza się dobrze przy małych tablicach, ale nie jest zalecany dla dużych zbiorów danych ze względu na gwałtowny wzrost czasu działania.

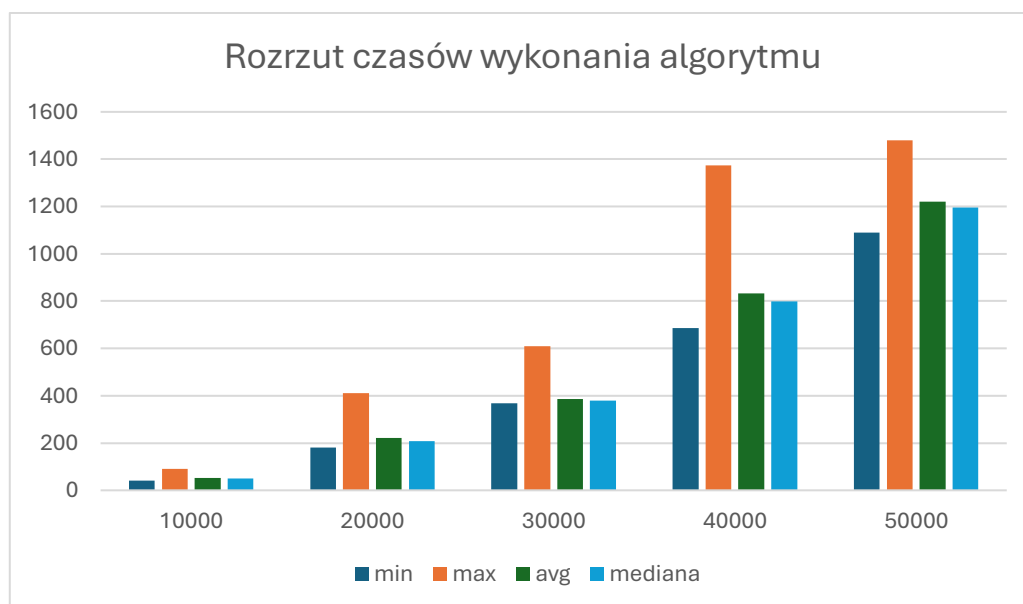
Binarne

Liczba elementów	10000	20000	30000	40000	50000
Min [ms]	41	181	369	685	1090
Max [ms]	90	412	610	1373	1479
Średnia [ms]	51,99	222,06	387	831,66	1220,3
Mediana [ms]	49	207	380	798	1196

Tabela 2. Czas działania algorytmu Binary Insertion Sort w zależności od liczby elementów

Wzrost czasu sortowania

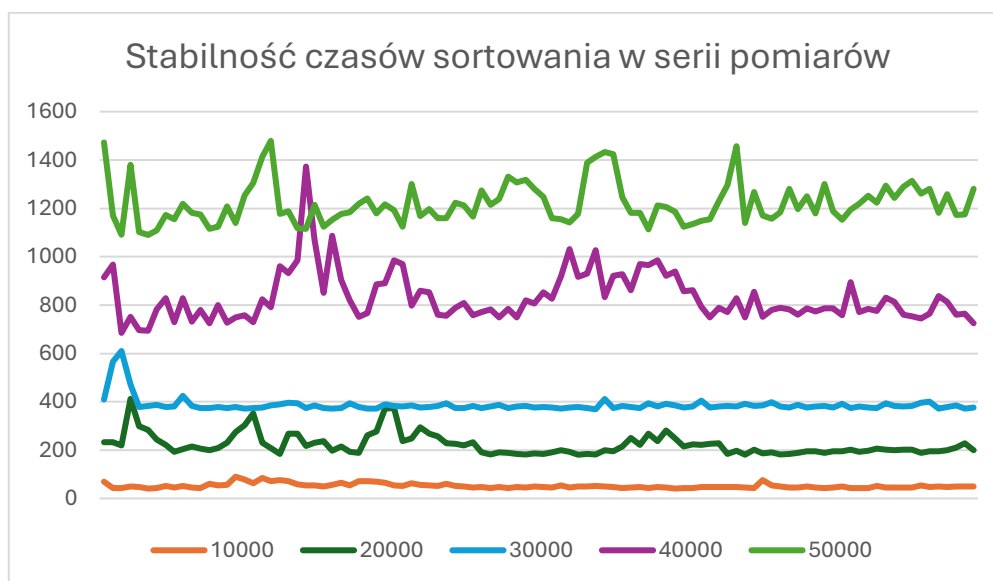
Z wykresu kolumnowego oraz danych zawartych w tabeli można jednoznacznie stwierdzić, że czas sortowania rośnie wraz z liczbą danych wejściowych. Dla 10 000 elementów średni czas sortowania wynosił około 52 ms, natomiast dla 50 000 elementów był to już czas rzędu 1220 ms. Choć Binary Insertion Sort również posiada złożoność $O(n^2)$, zastosowanie wyszukiwania binarnego pozwala na skrócenie czasu potrzebnego do odnalezienia miejsca wstawienia elementu, dzięki czemu ogólny czas sortowania jest niższy niż w przypadku klasycznego Insertion Sort.



Wykres 3. Rozrzut czasów wykonania algorytmu Binary Insertion Sort w zależności od ilości danych

Stabilność pomiarów

Podobnie jak w przypadku zwykłego Insertion Sort, również tutaj czasy sortowania są bardzo stabilne dla danej liczby elementów. Przykładowo, dla 30 000 elementów czasy mieszczą się w zakresie od 369 do 610 ms, a dla 50 000 od 1090 do 1479 ms. Wskazuje to na powtarzalność działania algorytmu oraz niewielki wpływ przypadkowości danych wejściowych.



Wykres 4. Stabilność czasów sortowania w serii pomiarów algorytmu Binary Insertion Sort w zależności od ilości danych

Wydajność dla małych i dużych zbiorów

Binary Insertion Sort oferuje zauważalną poprawę wydajności względem klasycznej wersji, szczególnie dla średnich i dużych rozmiarów danych. Dla mniejszych zbiorów (10 000–20 000) sortowanie przebiega bardzo sprawnie (poniżej 230 ms), natomiast dla większych tablic (40 000–50 000) czas działania osiąga wartości powyżej 1 sekundy, ale wciąż niższe niż dla zwykłego Insertion Sort.

Wnioski

- Binary Insertion Sort osiąga lepszą wydajność niż klasyczny Insertion Sort dzięki zastosowaniu binarnego wyszukiwania.
- Czas sortowania rośnie szybciej niż liniowo, ale wolniej niż w zwykłym Insertion Sort, mimo tej samej złożoności obliczeniowej.
- Algorytm jest stabilny i daje powtarzalne wyniki.
- Jest odpowiedni dla małych i średnich tablic, ale dla bardzo dużych danych warto rozważyć algorytmy o niższej złożoności, np. Quick Sort lub Heap Sort.

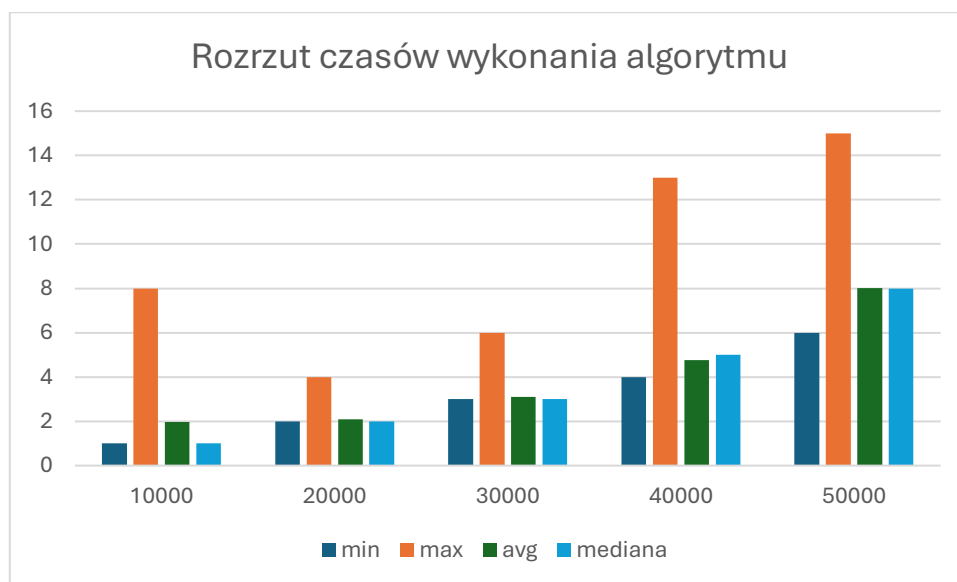
2. Sortowanie szybkie

Liczba elementów	10000	20000	30000	40000	50000
Min [ms]	1	2	3	4	6
Max [ms]	8	4	6	13	15
Średnia [ms]	1,96	2,08	3,11	4,77	8,02
Mediana [ms]	1	2	3	5	8

Tabela 3. Czas działania algorytmu Quick Sort w zależności od liczby elementów

Wzrost czasu sortowania

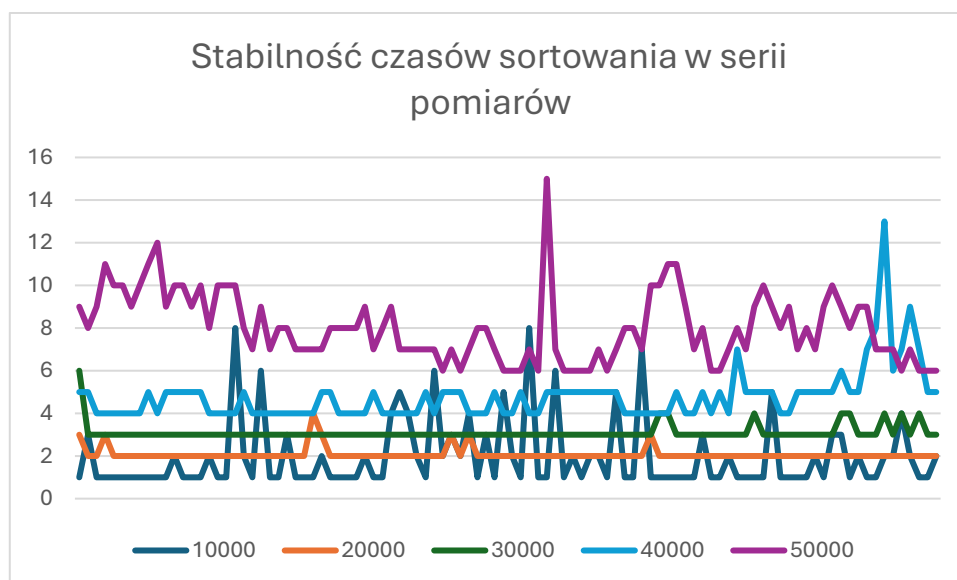
Z wykresu kolumnowego oraz analizy tabeli widać, że czas działania Quick Sort rośnie bardzo powoli wraz ze wzrostem rozmiaru danych. Średni czas sortowania dla 10 000 elementów wynosi około 2 ms, natomiast dla 50 000 elementów – zaledwie 8 ms. Porównując to z klasycznym Insertion Sort, widoczna jest ogromna przewaga Quick Sort pod względem wydajności. Taki wynik jest zgodny z teoretyczną złożonością algorytmu — $O(n \log n)$, co przekłada się na wysoką efektywność nawet dla dużych zbiorów.



Wykres 5. Rozrzut czasów wykonania algorytmu Quick Sort w zależności od ilości danych

Stabilność pomiarów

Algorytm Quick Sort charakteryzuje się bardzo niewielką zmiennością wyników dla danego rozmiaru danych. Przykładowo, dla 30 000 elementów czasy mieszczą się w zakresie 3–6 ms. Dla większych tablic (np. 50 000) różnice są już większe – od 6 do 15 ms – lecz dalej w granicach stabilności. Przedstawiona rozpiętość czasów świadczy o powtarzalności i stabilności działania algorytmu, niezależnie od konkretnego ułożenia danych wejściowych.



Wykres 6. Stabilność czasów sortowania w serii pomiarów algorytmu Quick Sort w zależności od ilości danych

Wydajność dla małych i dużych zbiorów

Quick Sort osiąga świetne wyniki niezależnie od wielkości danych. Zarówno dla mniejszych tablic (10 000–20 000), jak i większych (40 000–50 000) czas sortowania utrzymuje się na bardzo niskim poziomie – poniżej 10 ms. Takie wyniki czynią ten algorytm najbardziej efektywnym spośród wszystkich testowanych metod, a jednocześnie idealnym do zastosowań wymagających przetwarzania dużych wolumenów danych.

Podsumowanie tabelaryczne czasów wykonania:

Wnioski

- Quick Sort zapewnia zdecydowanie najlepszą wydajność spośród testowanych algorytmów, niezależnie od rozmiaru tablicy.
- Czas wykonania rośnie bardzo powoli, co wynika z efektywnej złożoności obliczeniowej $O(n \log n)$.
- Niska rozpiętość czasów świadczy o stabilności działania algorytmu.
- Algorytm ten doskonale nadaje się do sortowania dużych zbiorów danych w aplikacjach wymagających wysokiej wydajności.

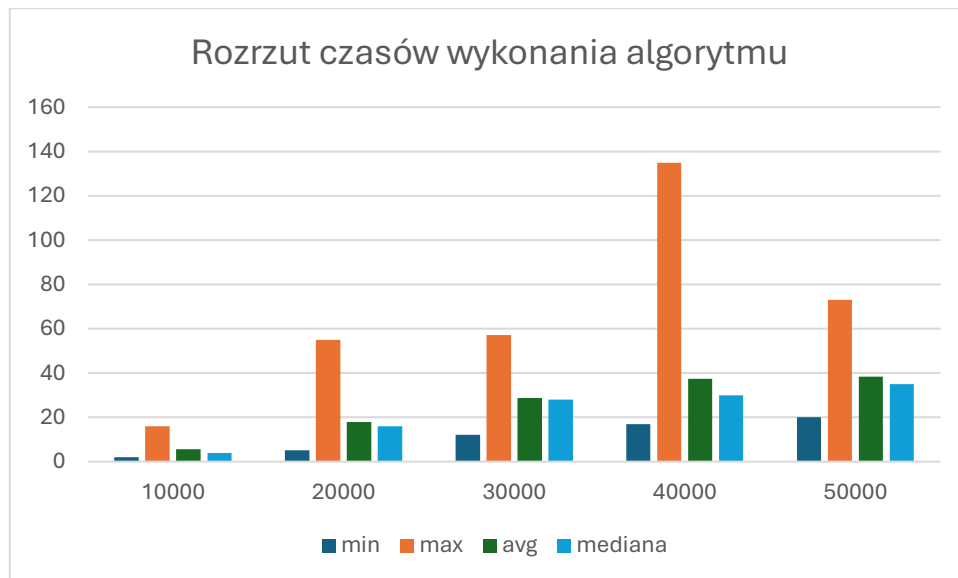
3. Sortowanie przez kopcowanie

Liczba elementów	10000	20000	30000	40000	50000
Min [ms]	2	5	12	17	20
Max [ms]	16	55	57	135	73
Średnia [ms]	5,53	17,78	28,68	37,38	38,3
Mediana [ms]	4	16	28	30	35

Tabela 4. Czas działania algorytmu Heap Sort w zależności od liczby elementów

Wzrost czasu sortowania

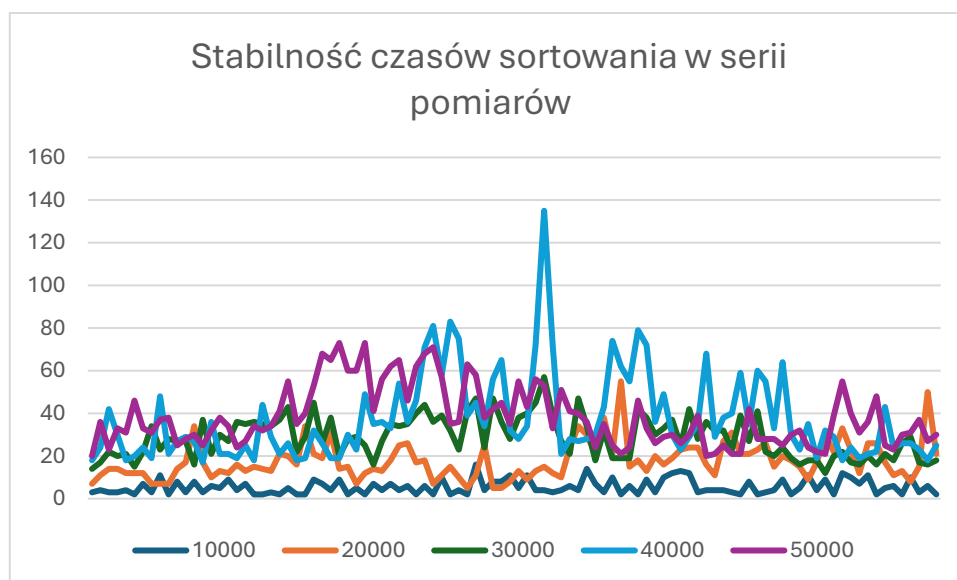
Z przedstawionej tabeli oraz wykresu kolumnowego wynika, że czas sortowania metodą kopcowania rośnie wraz ze wzrostem liczby elementów. Dla najmniejszego zestawu (10 000 elementów) średni czas sortowania wynosi około 5,5 ms, natomiast dla największego (50 000 elementów) wzrasta do około 38,3 ms. Wzrost ten jest jednak znacznie mniej gwałtowny niż w przypadku algorytmów o złożoności $O(n^2)$, co jest zgodne z teoretyczną złożonością Heap Sort – $O(n \log n)$.



Wykres 7. Rozrzut czasów wykonania algorytmu Heap Sort w zależności od ilości danych

Stabilność pomiarów

W przeciwieństwie do Quick Sort, Heap Sort wykazuje większą zmienność wyników dla tych samych rozmiarów danych. Przykładowo, dla 40 000 elementów czasy wahają się od 17 do aż 135 ms. Takie rozbieżności są widoczne również na wykresie liniowym, gdzie wartości poszczególnych pomiarów mogą odchyłać się istotnie od siebie. Może to wynikać z charakterystyki operacji zamiany w strukturze kopca oraz wpływu szczegółów implementacyjnych i układu danych wejściowych.



Wykres 8. Stabilność czasów sortowania w serii pomiarów algorytmu Heap Sort w zależności od ilości danych

Wydajność dla małych i dużych zbiorów

Heap Sort radzi sobie dobrze zarówno z małymi, jak i większymi tablicami. Dla 10 000–30 000 elementów czasy są niskie i stabilne. Dla większych zbiorów, mimo pewnej zmienności, czas sortowania pozostaje wciąż na akceptowalnym poziomie. Algorytm ten, dzięki swojej deterministycznej strukturze, dobrze sprawdza się tam, gdzie wymagana jest przewidywalna efektywność w czasie zbliżonym do $O(n \log n)$, niezależnie od danych wejściowych.

Wnioski

- Czas wykonania Heap Sort rośnie umiarkowanie wraz z liczbą elementów, zgodnie z jego złożonością $O(n \log n)$.
- Algorytm jest znacznie szybszy niż Insertion Sort, choć nieco wolniejszy i mniej stabilny niż Quick Sort.
- Widoczna zmienność wyników dla większych tablic może wynikać z charakterystyki zarządzania kopcem i kosztów zamian.
- Heap Sort jest dobrą alternatywą przy potrzebie stabilnej złożoności czasowej niezależnie od danych wejściowych, jednak może mieć mniej przewidywalne czasy wykonania w porównaniu do Quick Sort.

Wpływ początkowego rozkładu danych

W celu zbadania wpływu rodzaju uporządkowania danych wejściowych na wydajność algorytmów sortowania, przeprowadzono pomiary czasu wykonania dla tablic zawierających 30 000 elementów typu int, przygotowanych w różnych wariantach początkowego rozkładu danych (posortowane rosnąco, malejąco, w 33%, w 66% oraz nieposortowane). Dla każdego

typu uporządkowania wykonano serię testów, a uzyskane czasy przedstawiono w tabeli statystyk obejmującej wartości minimalne, maksymalne, średnie i mediany (w milisekundach).

1. Sortowanie przez wstawianie

Zwykle

Liczba elementów	ASCENDING	DESCENDING	PARTIALLY_33	PARTIALLY_66	RANDOM
Min [ms]	0	1171	518	324	575
Max [ms]	1	1670	665	719	612
Średnia [ms]	0.01	1299.76	555.57	342.17	585,32
Mediana [ms]	0	1309	548	333	584

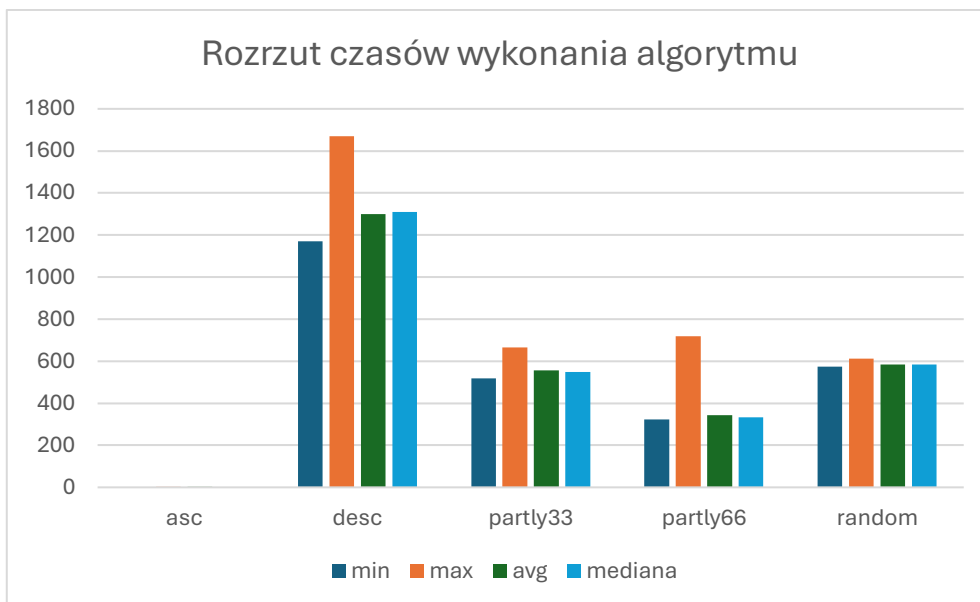
Tabela 5. Czas działania algorytmu Insertion Sort w zależności od rozkładu danych

Wpływ uporządkowania danych na czas sortowania

Z danych zawartych w tabeli oraz wykresu kolumnowego można jednoznacznie stwierdzić, że czas działania Insertion Sort silnie zależy od stopnia uporządkowania danych wejściowych. Najkrótszy czas został osiągnięty dla danych posortowanych rosnąco — wynosił on zaledwie około 0,01 ms, co odpowiada sytuacji najlepszej możliwej ($O(n)$). Algorytm nie musi wówczas wykonywać żadnych przesunięć elementów.

W przypadku danych całkowicie odwróconych (DESCENDING), czas sortowania był najwyższy i wyniósł średnio 1300 ms. Taki wynik jest zgodny z oczekiwaniami teoretycznymi — algorytm musi przesuwać wszystkie poprzednie elementy przy każdym wstawieniu, co daje złożoność $O(n^2)$.

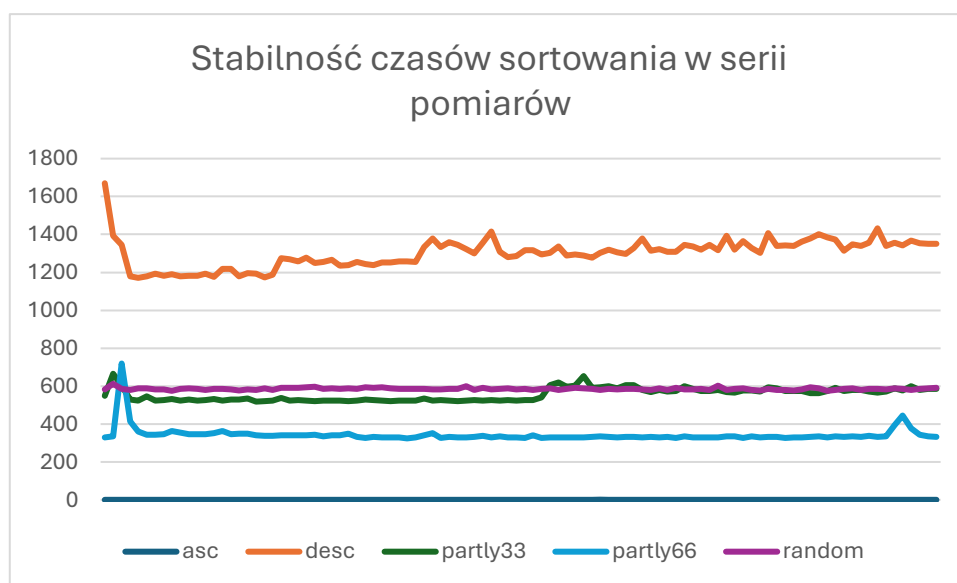
Dane częściowo uporządkowane (w 33% i 66%) również miały wpływ na wydajność — im więcej danych było początkowo na właściwym miejscu, tym szybciej działał algorytm. Dla 66% uporządkowania średni czas wynosił 342 ms, a dla 33% – 555,6 ms. Wyniki te pokazują, że nawet niepełne uporządkowanie może znacząco skrócić czas działania.



Wykres 9. Rozrzut czasów wykonania algorytmu Insertion Sort w zależności od rozkładu danych

Stabilność pomiarów

Dla danych uporządkowanych rosnąco (ASCENDING), czas wykonania był praktycznie zerowy i identyczny we wszystkich pomiarach, co świadczy o pełnej stabilności. Dla danych losowych i częściowo uporządkowanych rozrzuty czasów były niewielkie, co sugeruje, że Insertion Sort jest dość stabilny także w tych przypadkach. Jedynie dla danych odwróconych (DESCENDING) rozpiętość między czasem minimalnym (1171 ms) a maksymalnym (1670 ms) była zauważalna, co wiąże się z lokalizacją najbardziej kosztownych przesunięć w różnych przebiegach.



Wykres 10. Stabilność czasów sortowania w serii pomiarów algorytmu Insertion Sort w zależności od rozkładów danych

Wydajność w zależności od stopnia uporządkowania

Insertion Sort działa bardzo szybko dla danych prawie lub całkowicie uporządkowanych. Przykład z danymi uporządkowanymi rosnąco pokazuje, że algorytm w takich przypadkach osiąga czas liniowy. Wraz ze wzrostem nieuporządkowania danych rośnie liczba wymaganych przesunięć, co skutkuje gwałtownym wzrostem czasu działania. Dla danych całkowicie odwróconych sortowanie może trwać ponad 1,5 sekundy, co czyni ten algorytm nieefektywnym dla takich przypadków.

Wnioski

- Insertion Sort jest bardzo wrażliwy na początkowy układ danych – jego wydajność może zmieniać się o rzędy wielkości w zależności od rozkładu wejścia.
- Dane posortowane rosnąco prowadzą do czasu liniowego i minimalnej liczby operacji.
- Dane częściowo uporządkowane znacznie poprawiają wydajność w porównaniu z przypadkiem losowym.
- Dane odwrócone powodują skrajnie nieefektywne działanie — w najgorszym przypadku $O(n^2)$.
- Algorytm sprawdza się dobrze dla małych i prawie uporządkowanych zbiorów, ale nie nadaje się do dużych i chaotycznych danych.

Binarne

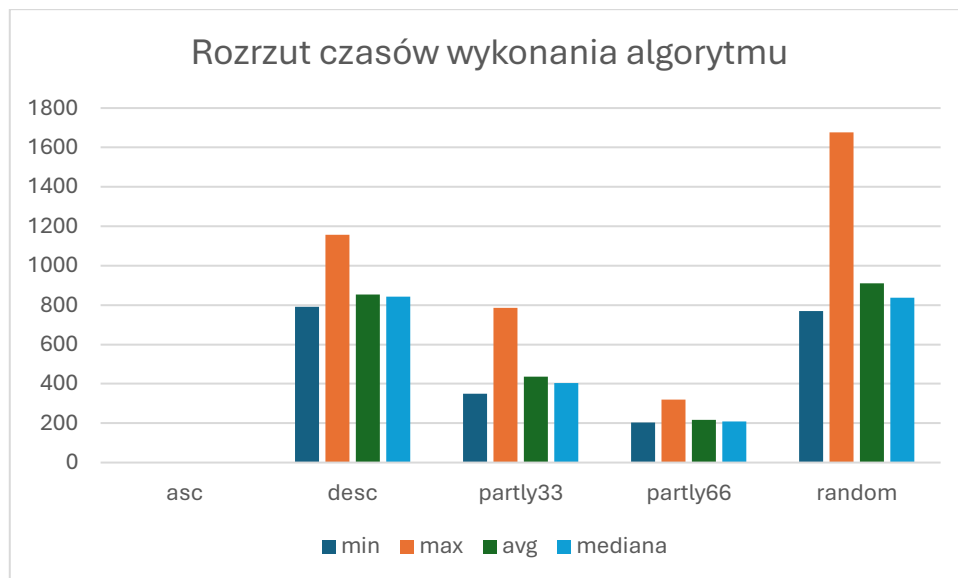
Liczba elementów	ASCENDING	DESCENDING	PARTIALLY_33	PARTIALLY_66	RANDOM
Min [ms]	1	790	349	203	768
Max [ms]	2	1157	786	320	1677
Średnia [ms]	1,1	852,3	437,29	217,34	909,98
Mediana [ms]	1	843	403	209	836

Tabela 6. Czas działania algorytmu Binary Insertion Sort w zależności od rozkładu danych

Wzrost czasu sortowania

Z danych przedstawionych w tabeli oraz wykresu kolumnowego wynika, że Binary Insertion Sort zachowuje się podobnie do zwykłego Insertion Sort — najkorzystniejszy czas osiąga dla danych uporządkowanych rosnąco, a najgorszy dla losowych. Dla ASCENDING średni czas wynosił zaledwie 1,1 ms, co jest niemal natychmiastowe. Choć wyszukiwanie miejsca odbywa się przez binary search, nie trzeba wykonywać przesunięć — co znacząco przyspiesza wykonanie.

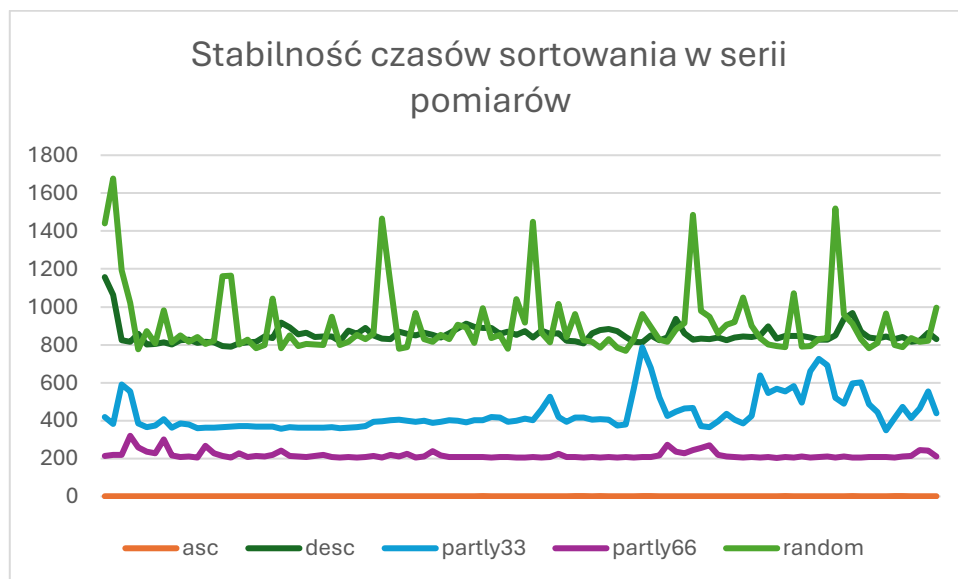
W przypadku DESCENDING, czas wzrasta do 852,3 ms, ale jest to nadal lepiej niż dla zwykłego Insertion Sort (~1300 ms). Wynika to z faktu, że binary search przyspiesza lokalizację miejsca wstawienia, ale przesuwanie elementów nadal musi być wykonane w całości.



Wykres 11. Rozrzut czasów wykonania algorytmu Binary Insertion Sort w zależności od rozkładu danych

Stabilność pomiarów

Pomiar czasu dla danych posortowanych rosnąco był bardzo stabilny — różnice wahały się w zakresie 1–2 ms. Dla danych losowych oraz częściowo uporządkowanych rozrzut również był umiarkowany i nie przekraczał kilkudziesięciu milisekund. W przypadku danych odwrotnych (DESCENDING) zauważono większe wahania — od 790 do 1157 ms — co może wynikać z lokalnych różnic w układzie danych. Największe wahania zauważono jednak w rozkładzie randomowym, ponieważ może mieć dane posortowane choćby częściowo jak i wcale.



Wykres 12. Stabilność czasów sortowania w serii pomiarów algorytmu Binary Insertion Sort w zależności od rozkładów danych

Wydajność w zależności od stopnia uporządkowania

Binary Insertion Sort bardzo dobrze reaguje na częściowe uporządkowanie danych. Dla 33% uporządkowania uzyskano średni czas 437 ms, a dla 66% – zaledwie 217 ms. Wyniki te świadczą o tym, że mniejsza liczba nieuporządkowanych elementów skutkuje krótszym czasem przesuwania, a binary search dodatkowo zmniejsza koszt wyszukiwania miejsca.

Dla danych całkowicie losowych algorytm osiągnął średni czas 910 ms. Jest to wynik pośredni — dłuższy niż dla danych częściowo uporządkowanych, ale krótszy niż dla danych odwrotnie posortowanych. Wskazuje to, że Binary Insertion Sort dobrze radzi sobie także w przypadkach neutralnych, gdzie nie występuje ani najlepszy, ani najgorszy przypadek.

Wnioski

- Binary Insertion Sort osiąga znacząco lepsze czasy wykonania niż zwykły Insertion Sort, szczególnie dla dużych i uporządkowanych danych.
- Dla posortowanych danych działa bardzo szybko (średnio ~ 1 ms), ponieważ wyszukiwanie binarne minimalizuje czas lokalizacji pozycji.
- Dla częściowo uporządkowanych danych Binary Insertion Sort działa bardzo sprawnie, co czyni go przydatnym w sytuacjach, gdzie dane są wstępnie posortowane.
- Algorytm zachowuje dobrą stabilność pomiarów, zwłaszcza dla korzystniejszych przypadków.

2. Sortowanie szybkie

Liczba elementów	ASCENDING	DESCENDING	PARTIALLY_33	PARTIALLY_66	RANDOM
Min [ms]	0	0	3	2	3
Max [ms]	2	2	22	20	6
Średnia [ms]	0,34	0,8	4,8	4,46	3,11
Mediana [ms]	0	1	3	3	3

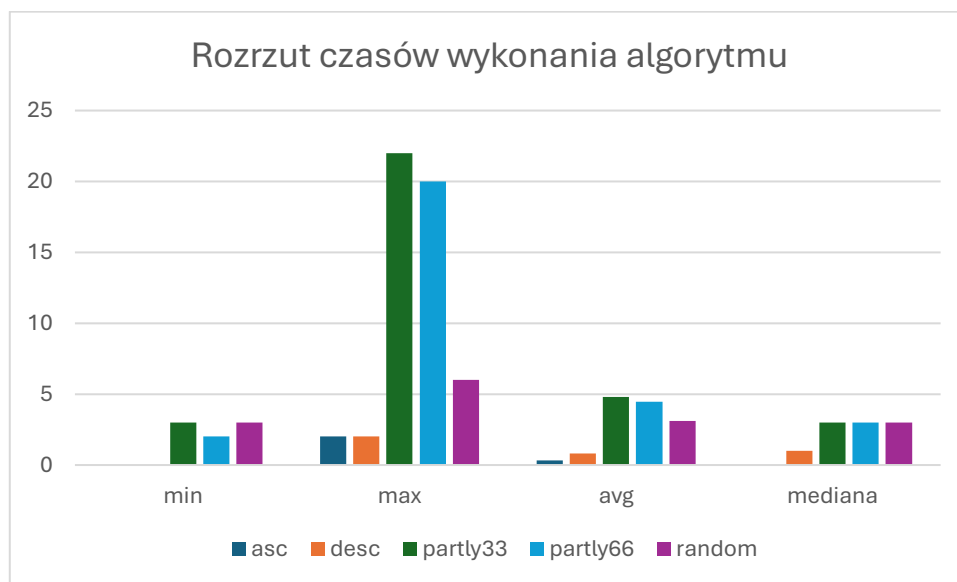
Tabela 7. Czas działania algorytmu Quick Sort w zależności od rozkładu danych

Wzrost czasu sortowania

Quick Sort okazał się zdecydowanie najszybszym algorytmem w zestawieniu. Dla danych rosnąco uporządkowanych (ASCENDING) średni czas sortowania wyniósł zaledwie 0,34 ms, a dla malejących (DESCENDING) — 0,8 ms. Bardzo niskie wartości wskazują na skuteczne działanie algorytmu przy takich danych oraz na efektywny wybór pivotów, który zapobiegł spadkowi wydajności do złożoności $O(n^2)$.

W przypadku danych częściowo uporządkowanych (PARTIALLY_33 i PARTIALLY_66) czasy były nieco wyższe — odpowiednio 4,8 ms i 4,46 ms — jednak nadal utrzymywały się na bardzo dobrym poziomie. Dla danych losowych (RANDOM) średni czas sortowania wyniósł 3,11 ms, co potwierdza, że Quick Sort dobrze radzi sobie z typowym, niezorganizowanym wejściem.

Warto zauważyć, że mimo okazjonalnych skoków czasu (np. PARTIALLY_33: max = 22 ms), algorytm działał szybko i stabilnie w większości przypadków. Wyniki te potwierdzają wysoką efektywność Quick Sorta w szerokim zakresie przypadków.

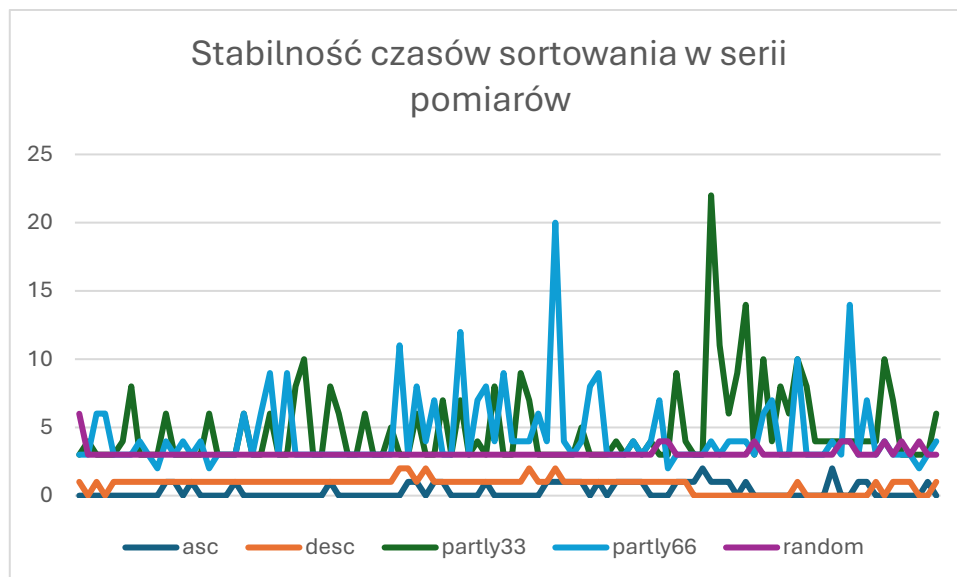


Wykres 13. Rozrzut czasów wykonania algorytmu Insertion Sort w zależności od rozkładu danych

Stabilność pomiarów

Quick Sort wykazał **bardzo wysoką stabilność** jedynie dla danych w pełni uporządkowanych:

- ASCENDING i DESCENDING — czasy sortowania niemal identyczne (zwykle 0–1 ms), z wyjątkowo małym rozrzutem,
- RANDOM — stabilność nadal wysoka, czasy głównie w zakresie 3 ms,
- PARTIALLY_66 i PARTIALLY_33 — charakteryzowały się większą zmiennością czasów wykonania. Choć wartości średnie pozostały stosunkowo niskie, pojedyncze pomiary przekraczały 10 ms, a w skrajnych przypadkach nawet 20 ms. Przyczyną tych odchyłeń jest fakt, że wybierany pivot — środkowy element — nie zawsze trafnie odzwierciedla medianę danych, co skutkuje nierównomiernym podziałem tablicy.



Wykres 13. Stabilność czasów sortowania w serii pomiarów algorytmu Quick Sort w zależności od rozkładów danych

Wnioski

- Quick Sort niezależnie od uporządkowania danych działa bardzo szybko, nawet przy częściowym lub całkowitym braku porządku.
- Dla danych losowych i uporządkowanych osiąga rekordowo niskie czasy sortowania, nawet poniżej 1 ms.
- Cechuje się dużą stabilnością wyników, co czyni go niezawodnym wyborem do sortowania danych ogólnego przeznaczenia.
- Spośród wszystkich analizowanych algorytmów, Quick Sort osiąga najlepszy kompromis między szybkością, stabilnością i odpornością na dane wejściowe, co czyni go najwydajniejszym z badanych rozwiązań.

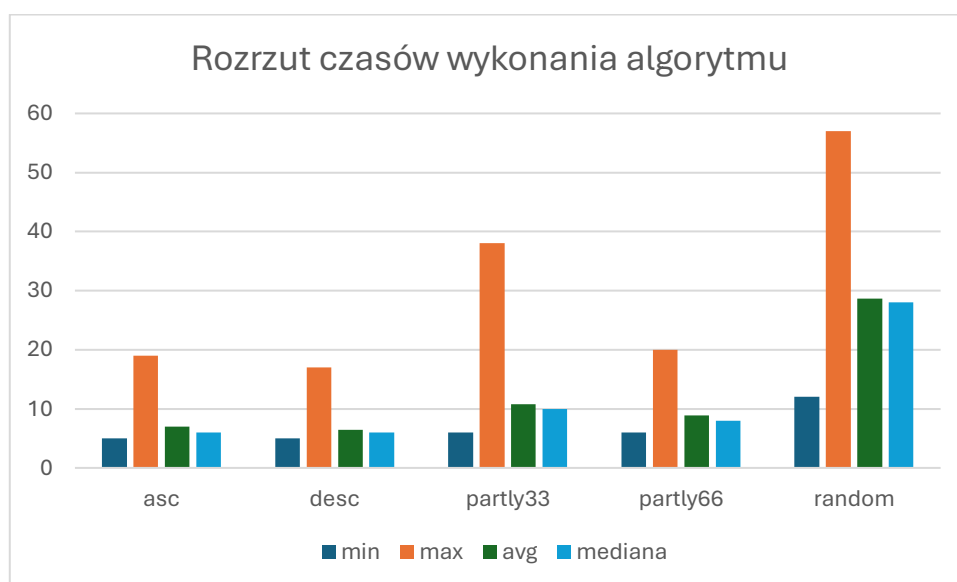
3. Sortowanie przez kopcowani

Liczba elementów	ASCENDING	DESCENDING	PARTIALLY_33	PARTIALLY_66	RANDOM
Min [ms]	5	5	6	6	12
Max [ms]	19	17	38	20	57
Średnia [ms]	7	6,47	10,79	8,92	28,68
Mediana [ms]	6	6	10	8	28

Tabela 8. Czas działania algorytmu Heap Sort w zależności od rozkładu danych

Na podstawie zgromadzonych danych i wykresu kolumnowego można zauważyć wyraźną zależność między typem danych wejściowych a czasem wykonania algorytmu Heap Sort. W przypadku danych całkowicie losowych (RANDOM), średni czas sortowania wynosił aż 28,68 ms, co jest istotnie większą wartością w porównaniu do danych uporządkowanych (ASCENDING: 7 ms, DESCENDING: 6,47 ms).

Pomimo teoretycznej złożoności Heap Sort wynoszącej $O(n \log n)$ i ogólnej odporności na układ danych, różnice czasowe wskazują, że w praktyce dane losowe powodują intensywniejszą reorganizację kopca. W przypadkach częściowego uporządkowania (33% i 66%), czasy wykonania są pośrednie – dane PARTIALLY_33 miały średnio 10,79 ms, a PARTIALLY_66 – 8,92 ms, co sugeruje, że większy stopień uporządkowania pozytywnie wpływa na czas wykonania.

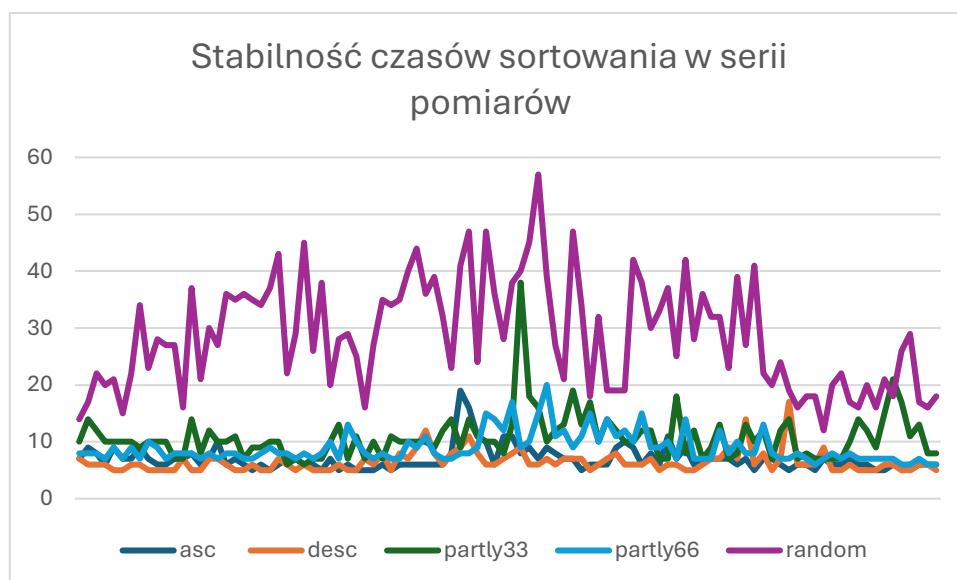


Wykres 14. Rozrzut czasów wykonania algorytmu Insertion Sort w zależności od rozkładu danych

Stabilność pomiarów

Stabilność działania Heap Sort jest wysoka w przypadkach danych uporządkowanych rosnąco i malejąco – wartości minimalne i maksymalne są bardzo bliskie sobie (np. DESCENDING: min = 5 ms, max = 7 ms). Wskazuje to, że algorytm działa przewidywalnie i nie podlega dużym fluktuacjom czasu, jeśli struktura danych wejściowych jest regularna.

Z kolei w przypadku danych losowych zauważalny jest znacznie większy rozrzut wyników (od 12 ms do 57 ms). Oznacza to, że losowość może prowadzić do niestabilności w czasie budowania kopca, co w niektórych przypadkach wymaga znacznie większej liczby operacji przestawiania elementów.



Wykres 14. Stabilność czasów sortowania w serii pomiarów algorytmu Heap Sort w zależności od rozkładów danych

Wydajność dla różnych zbiorów danych

Heap Sort wykazuje wysoką efektywność niezależnie od układu danych, jednak najlepsze wyniki osiąga przy danych uporządkowanych (zarówno rosnąco, jak i malejąco), ponieważ mimo że algorytm zawsze buduje kopiec, struktura danych może wpływać na liczbę przestawień w fazie sortowania.

Dla danych częściowo uporządkowanych algorytm wymaga więcej pracy niż dla danych w pełni uporządkowanych, ale mniej niż dla losowych – pokazuje to przykład: średni czas dla PARTIALLY_33 to prawie 11 ms, a dla PARTIALLY_66 już tylko 9 ms, co sugeruje, że większy stopień wstępnego uporządkowania przyspiesza wykonanie.

Dane całkowicie losowe stanowią największe obciążenie dla algorytmu, ze względu na konieczność ciągłej przebudowy kopca i częste operacje zamiany.

Wnioski

- Heap Sort jest odporny na początkowe uporządkowanie danych, jednak nie całkowicie obojętny. W praktyce różne rozkłady danych mają wpływ na szczegółową liczbę operacji w trakcie sortowania.
- Najlepszą wydajność uzyskano dla danych posortowanych malejąco, a najgorszą – dla danych losowych.
- Dane częściowo uporządkowane (zwłaszcza 66%) są wystarczająco „pomocne” dla algorytmu, by osiągnąć lepsze wyniki niż przy pełnym chaosie.

Wpływ typów danych

W celu zbadania wpływu typu danych wejściowych na wydajność algorytmów sortowania, przeprowadzono pomiary czasu wykonania dla tablic zawierających 30 000 elementów, trzech różnych typów: char, int oraz float. Dla każdego z typów wykonano serię testów, a uzyskane wyniki zestawiono w tabeli zawierającej minimalny, maksymalny, średni i medianę czasu wykonania (w milisekundach).

Typ danych	Minimum [ms]	Maksimum [ms]	Średnia [ms]	Mediana [ms]
char	2	6	2,29	2
float	3	14	4,47	4
int	3	6	3,11	3

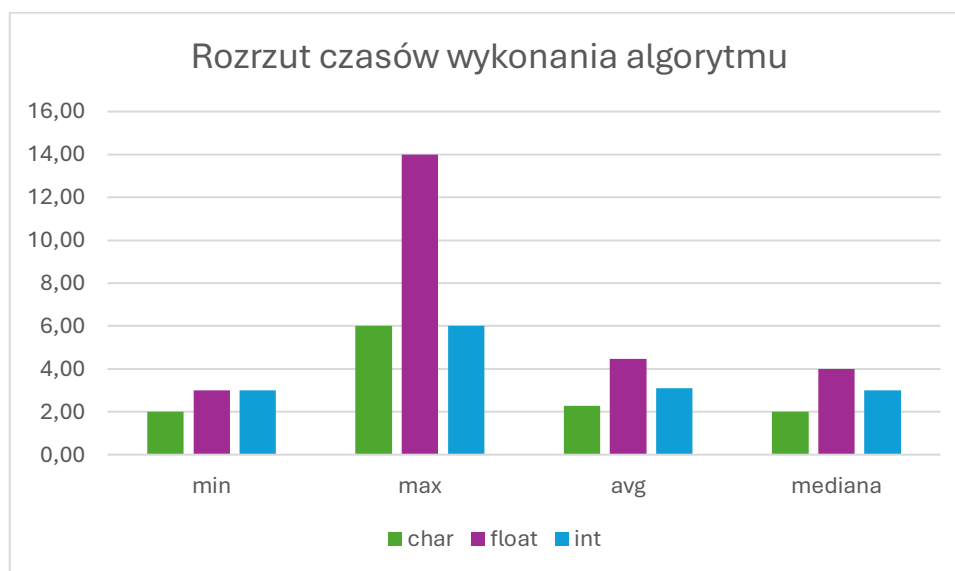
Tabela 9. Czas działania algorytmu Quick Sort w zależności od typu danych

Porównanie wydajności względem typu danych

Typ char wypada najszybciej: Zarówno średni czas (2,29 ms), jak i mediana (2,00 ms), a także bardzo niskie wartości minimalne i maksymalne świadczą o tym, że sortowanie znaków jest najbardziej efektywne czasowo. Wynika to prawdopodobnie z niewielkiego rozmiaru danych oraz prostoty ich porównań.

Typ float jest wyraźnie najwolniejszy: Najwyższy czas średni (4,47 ms) oraz maksymalny (14,00 ms) sugerują, że liczby zmiennoprzecinkowe są najtrudniejsze do posortowania. Może to wynikać z większej złożoności operacji porównywania oraz reprezentacji binarnej wartości typu float.

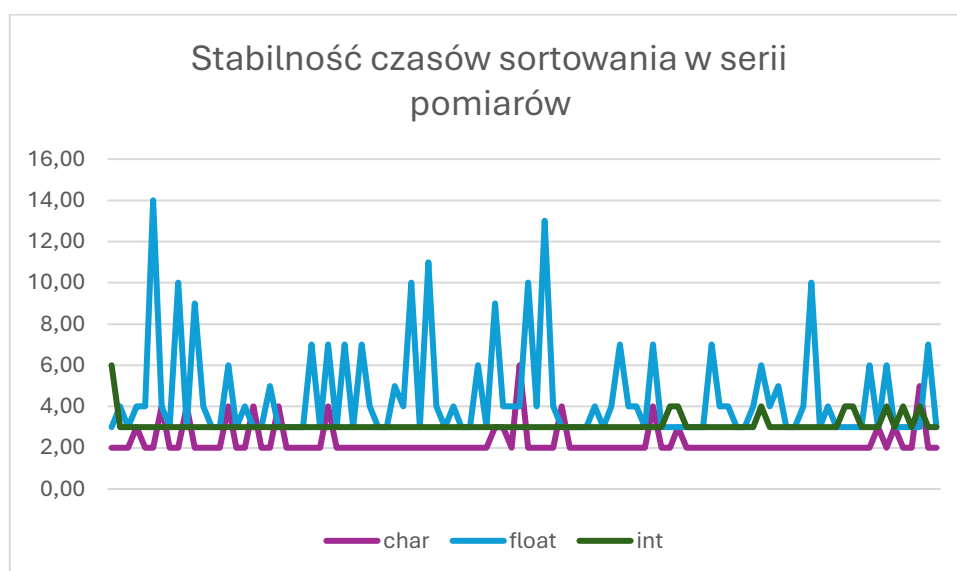
Typ int osiąga wyniki pośrednie: Czas sortowania dla typu int jest nieco wyższy niż dla char, ale znacznie niższy niż dla float. Średni czas (3,11 ms) oraz mediana (3,00 ms) pozostają na stabilnym poziomie i wskazują na dobrą wydajność.



Wykres 15. Rozrzut czasów wykonania algorytmu Quick Sort w zależności od typu danych

Stabilność pomiarów

Analiza stabilności pomiarów wskazuje na względnie niewielkie wahania czasów wykonania w obrębie każdego z typów danych. Dla typu char zróżnicowanie czasów między pomiarami jest minimalne więc można stwierdzić, że algorytm działa bardzo równomiernie przy tym typie danych. Podobnie jest dla typu int czasy są bardzo stabilne. Typ float ma większe rozpiętości, co sugeruje pewne niestabilności, wynika to z trudności związanych z reprezentacją liczb zmiennoprzecinkowych oraz bardziej złożoną naturą operacji porównywania tych liczb, ale nadal mieści się w granicach akceptowalnej stabilności.



Wykres 16. Stabilność czasów sortowania w serii pomiarów algorytmu Quick Sort w zależności od typu danych

Różnice między typami są istotne

Mimo że algorytm Quick Sort teoretycznie powinien działać niezależnie od typu danych, w praktyce widoczne są znaczące różnice. Reprezentacja danych wpływa na czas działania — im bardziej złożony typ, tym dłuższy czas sortowania.

Wnioski

- Typ char zapewnia najwyższą wydajność sortowania dzięki prostocie danych.
- Typ float jest najmniej wydajny, co może być istotne przy sortowaniu dużych zbiorów danych zmiennoprzecinkowych.
- Różnice wynikają ze sposobu reprezentacji i porównywania danych — efektywność algorytmu zależy nie tylko od jego struktury, ale i od charakterystyki danych wejściowych.

Wersja pijanego studenta

1. Wpływ stopnia pijaństwa

W testach algorytmu *drunk insertion sort* zastosowano wartości parametru „pijaństwa” na poziomie 10%, 25%, 50%, 75% oraz 90%. Celem testu było zbadanie wpływu tych wartości na czas wykonania algorytmu oraz jego wydajność przy różnych poziomach zaburzenia poprawności działań algorytmu.

Wzrost czasu sortowania przy różnych poziomach „pijaństwa”

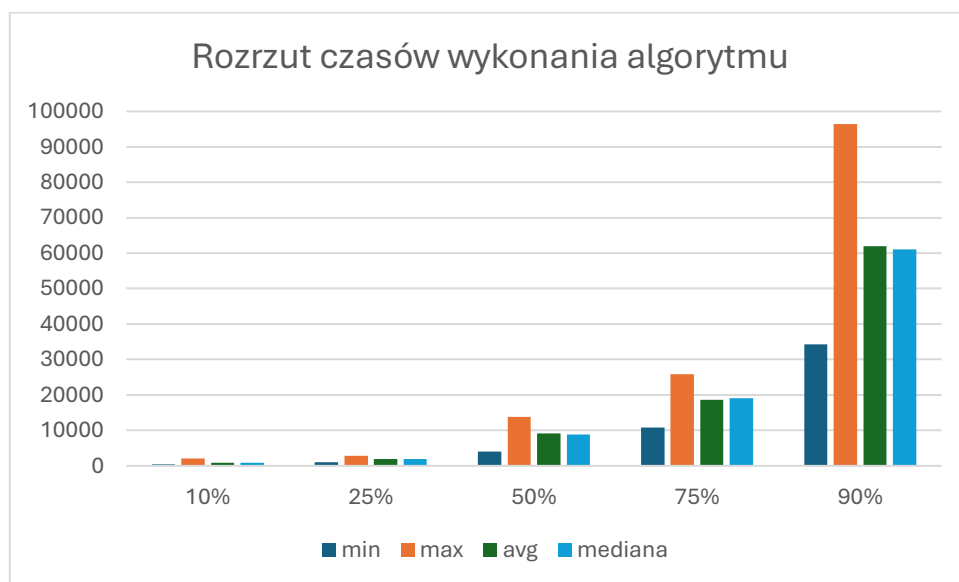
Z wyników testów wynika, że parametry pijaństwa mają bardzo silny wpływ na czas wykonania algorytmu. Im wyższy poziom pijaństwa, tym więcej błędów w operacjach algorytmu, co znacząco zwiększa czas potrzebny na sortowanie tablicy.

Liczba elementów	10%	25%	50%	75%	90%
Min [ms]	445	1035	3976	10745	34311
Max [ms]	2080	2799	13764	25839	96346
Średnia [ms]	929,28	1972,59	9085,55	18628,3	62018,9
Mediana [ms]	917	1982	8837	19130	61031

Tabela 10. Czas działania algorytmu Drunk Insertion Sort w zależności od stopnia pijaństwa

Wzrost czasu sortowania w zależności od poziomu pijaństwa

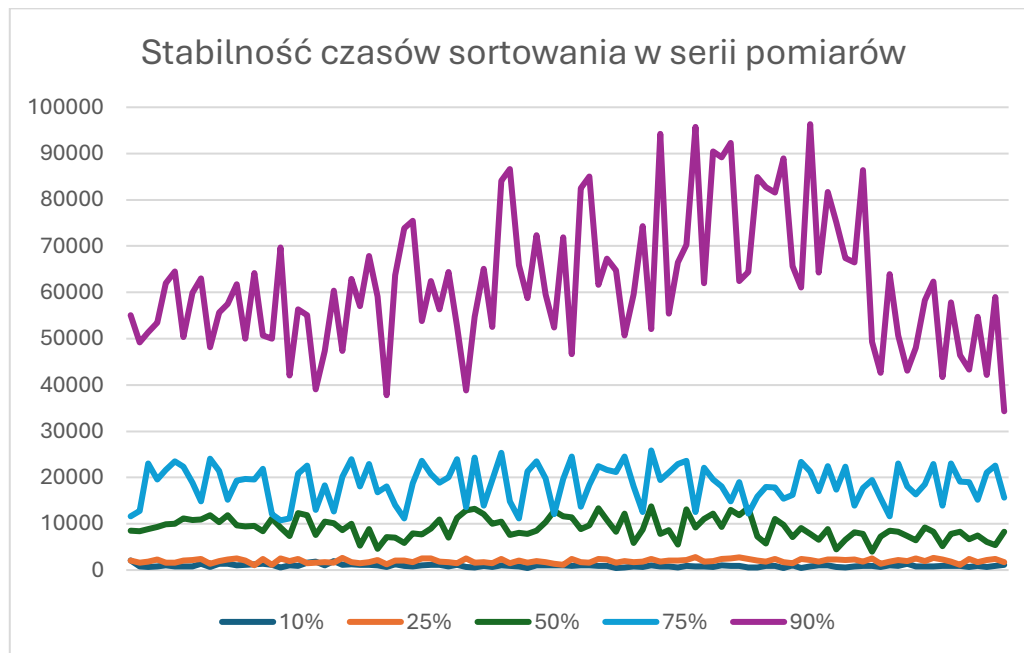
Z wykresu kolumnowego oraz przedstawionych danych widać wyraźnie, jak dramatycznie rośnie czas wykonania algorytmu w miarę wzrostu poziomu pijaństwa. Przy 10% pijaństwa algorytm działa stosunkowo sprawnie, przypominając klasyczny algorytm sortowania przez wstawianie, jednak już przy 90% czas działania znacznie przekracza minutę, co wskazuje na bardzo dużą liczbę błędów wymagających wielokrotnych poprawek.



Wykres 17. Rozrzut czasów wykonania algorytmu Drunk Insertion Sort w zależności od stopnia pijaństwa

Stabilność pomiarów

W miarę jak rośnie poziom pijaństwa, stabilność pomiarów spada. Na przykład, przy 10% pijaństwa różnice między minimalnym a maksymalnym czasem wykonania są stosunkowo niewielkie (445 ms do 2080 ms), co wskazuje na większą stabilność działania algorytmu. Jednak przy wyższych poziomach pijaństwa, szczególnie przy 90%, rozpiętości czasów rosną drastycznie, co potwierdza, że algorytm staje się coraz bardziej niestabilny.



Wykres 18. Stabilność czasów sortowania w serii pomiarów algorytmu Drunk Insertion Sort w zależności od stopnia pijaństwa

Wydajność dla różnych poziomów pijaństwa

- **Przy 10% pijaństwa** algorytm działa zbliżony do tradycyjnego *insertion sort*, z minimalnymi opóźnieniami, co czyni go nadal wydajnym, choć z drobnymi zaburzeniami.
- **Przy 25% pijaństwa** czas działania niemal się podwaja, co wskazuje na większą liczbę błędnych operacji i konieczność wykonania więcej iteracji.
- **Przy 50% pijaństwa** algorytm staje się wyraźnie mniej efektywny, z czasem wykonania przekraczającym 9 sekund. Prawdopodobieństwo błędów i konieczność ich poprawiania prowadzą do poważnych trudności w sortowaniu.
- **Przy 75% pijaństwa** czas sortowania wzrasta do 18,6 sekund, a algorytm wymaga wielu powtórzeń, aby osiągnąć poprawną kolejność.

- **Przy 90% pijaństwa** algorytm staje się ekstremalnie wolny, z czasem wykonania przekraczającym 1 minutę. Zmniejszenie efektywności jest skrajne, co sprawia, że algorytm działa w sposób niepraktyczny.

Wnioski

- Dla większej ilości elementów oraz $90\% <$ pijaństwa algorytm wykonywałby się na tyle długo że musiałabym poświęcić miesiąc na testowanie
- Dla 0% student jest w pełni trzeźwy więc od razu rozwiązuje poprawnie insertion sort, zaś dla 100% student od ilości alkoholu we krwi "idzie spać" i pomija całkowicie wszystkie elementy.
- Eksperyment potwierdza, że nawet niewielkie zaburzenie w poprawności algorytmu może mieć ogromny wpływ na czas wykonania i sprawność algorytmu w zadaniach o większym stopniu trudności.

2. Wpływ początkowego rozkładu danych

W celu zbadania wpływu rodzaju uporządkowania danych wejściowych na wydajność algorytmu „drunk insertion sort”, przeprowadzono pomiary czasu wykonania dla tablic zawierających 10 000 elementów typu int, przygotowanych w różnych wariantach początkowego rozkładu danych(posortowane rosnąco, malejąco, częściowo uporządkowane w 33%, częściowo uporządkowane w 66%, oraz dane losowe). Dla każdego typu uporządkowania wykonano serię testów, a uzyskane czasy przedstawiono w tabeli statystyk obejmującej wartości minimalne, maksymalne, średnie i mediany (w milisekundach).

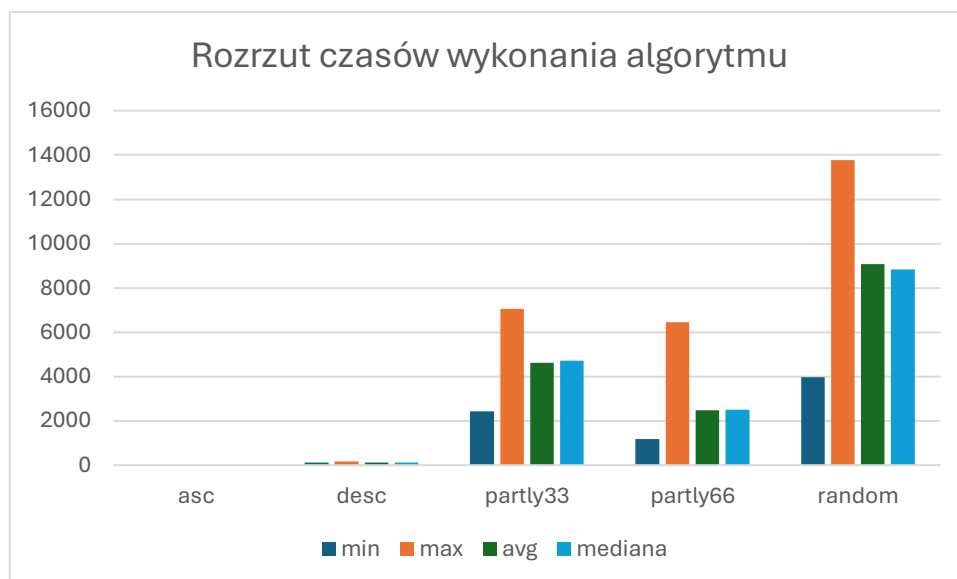
Liczba elementów	ASCENDING	DESCENDING	PARTIALLY_33	PARTIALLY_66	RANDOM
Min [ms]	0	110	2440	1188	3976
Max [ms]	0	172	7055	6449	13764
Średnia [ms]	0	119,71	4633,23	2477,86	9085.55
Mediana [ms]	0	116	4717	2496	8837

Tabela 11. Czas działania algorytmu Drunk Insertion Sort w zależności od rozkładu danych

Wzrost czasu sortowania przy różnych układach danych

Z wyników testów wynika, że układ danych ma kluczowy wpływ na czas wykonania algorytmu. Im bardziej uporządkowane dane, tym szybciej algorytm dochodzi do poprawnego wyniku, nawet przy obecności błędów. Z kolei dane w pełni losowe powodują znaczne wydłużenie czasu wykonania. Z wykresu kolumnowego oraz przedstawionych danych widać wyraźnie, jak rośnie czas wykonania algorytmu w zależności od układu danych. Przy danych

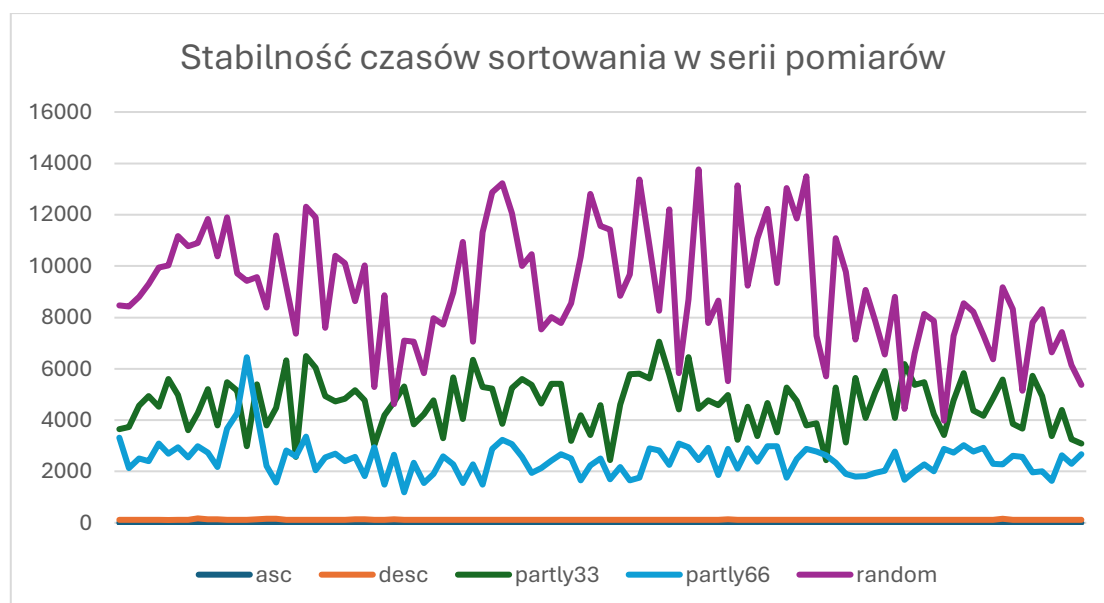
rosnąco posortowanych czas sortowania wynosi 0 ms, ponieważ algorytm nie musi wykonywać praktycznie żadnych operacji. W przypadku danych losowych czas wykonania osiąga prawie 9090 sekund w średnich wynikach, a maksymalny czas przekracza 13764 sekund, co wskazuje na znaczne trudności w uporządkowaniu danych.



Wykres 19. Rozrzut czasów wykonania algorytmu Drunk Insertion Sort w zależności od rozkładu danych

Stabilność pomiarów

W miarę jak układ danych staje się mniej uporządkowany, stabilność pomiarów maleje. Na przykład, dla danych rosnąco posortowanych czas wykonania jest jednolity (0 ms), co wskazuje na bardzo stabilną pracę algorytmu. Z kolei przy danych losowych rozpiętość czasów wykonania rośnie drastycznie, co potwierdza, że algorytm staje się niestabilny i wymaga wielu poprawek.



Wykres 20. Stabilność czasów sortowania w serii pomiarów algorytmu Drunk Insertion Sort w zależności od rozkładów danych

Wydajność dla różnych układów danych

- **Dane rosnąco posortowane:** Algorytm działa bardzo efektywnie, ponieważ dane są już uporządkowane, więc nie wymaga praktycznie żadnych operacji co student nawet pijany zauważa.
- **Dane malejąco posortowane:** Algorytm działa nieco wolniej choć spodziewalibyśmy się że to "najgorszy przypadek", tak jak jest to dla insertion sort. Dla pijanego studenta tak nie jest, bo tablica jest bardzo przewidywalna: największe liczby trzeba tylko przepchnąć w dół. Dzięki temu nawet przy 50% "pijaństwa" algorytm dosyć szybko osiąga poprawne sortowanie, a średni czas wykonania jest niski.
- **Dane częściowo uporządkowane (33%):** Zwiększenie stopnia zamieszania w danych skutkuje znacznie większym czasem sortowania. Algorytm wymaga wielu iteracji, aby osiągnąć poprawny rezultat.
- **Dane częściowo uporządkowane (66%):** Algorytm radzi sobie lepiej w porównaniu do danych losowych, ale nadal wymaga więcej operacji niż w przypadku danych rosnąco posortowanych.
- **Dane losowe:** Algorytm działa najwolniej, ponieważ musi wielokrotnie przechodzić przez dane, by je uporządkować. Wydajność spada dramatycznie w miarę wzrostu nieuporządkowania danych.

Wnioski

- Im mniej uporządkowane dane, tym dłużej trwa ich sortowanie.
- Student choć pijany zauważa pewne zależności jak pokazuje przykład DESCENDING
- Stopień początkowego uporządkowania danych odgrywa kluczową rolę w wydajności tego typu niestandardowego algorytmu sortowania.

3. Wpływ typów danych

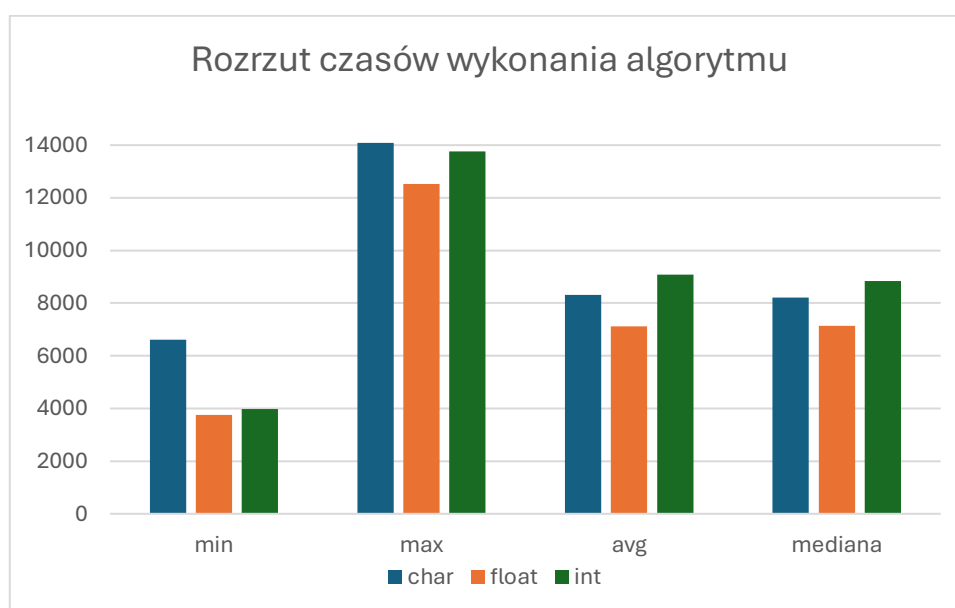
W przeprowadzonych badaniach zbadano wpływ typu danych na efektywność algorytmu pijanego studenta. Algorytm ten polega na losowym pomijaniu elementów w trakcie sortowania, co wpływa na czas wykonania operacji. W badaniu uwzględniono trzy typy danych: float, char oraz int. Dla każdego typu wykonano pomiary czasów wykonania sortowania, uwzględniając wartości minimalne, maksymalne, średnie oraz mediany. Poniżej przedstawiono tabelę wyników testów.

Typ danych	Minimum [ms]	Maksimum [ms]	Średnia [ms]	Mediana [ms]
float	3749	12523	7114,3	7132
char	6615	14084	8321,1	8211
int	3976	13764	9085,6	8837

Tabela 12. Czas działania algorytmu Drunk Insertion Sort w zależności od typu danych

Wzrost czasu sortowania w zależności od typu

Z wyników testów wynika, że czas sortowania zależy w dużym stopniu od typu danych. Typ float osiągnął najlepsze wyniki, zarówno pod względem średnich czasów, jak i mediany. Czas sortowania dla danych typu float wynosił średnio 7114,3ms, co można uznać za szybkie wykonanie w porównaniu do innych. W przypadku typu char, czasy były wyższe, osiągając średnią na poziomie 8321,1ms, co wskazuje na większą trudność w sortowaniu danych tego typu. Najgorsze wyniki uzyskano dla typu int, gdzie średni czas sortowania wyniósł 9085,6ms.

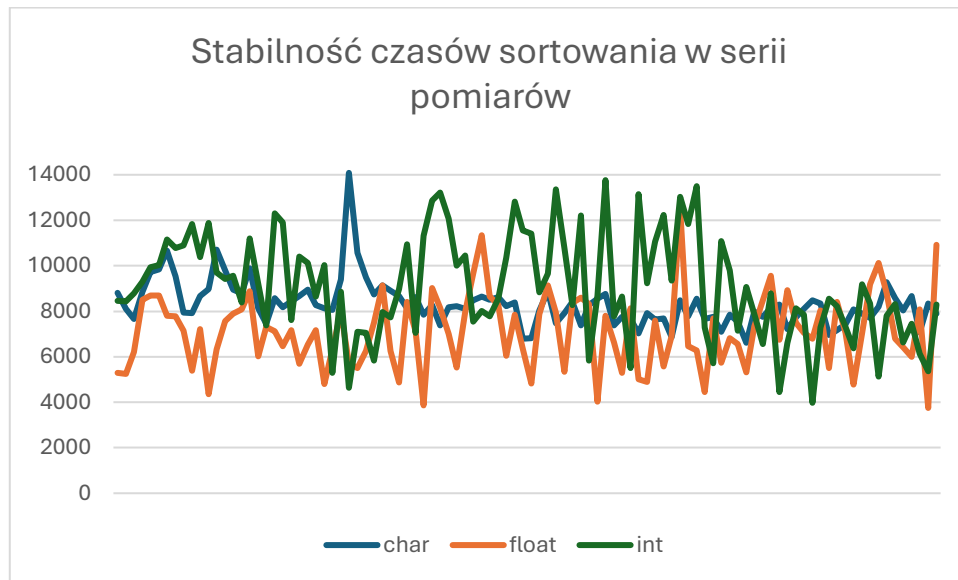


Wykres 21. Rozrzut czasów wykonania algorytmu Drunk Insertion Sort w zależności od typu danych

Stabilność sortowania

Wykres potwierdza, że dane typu char wykazują największą stabilność – linia przebiega stosunkowo równomiernie, z mniejszą liczbą gwałtownych skoków. Oznacza to, że czas sortowania tych danych jest bardziej przewidywalny. Dla typu float widać większe wahania w dolnym zakresie wartości, ale bez skrajnych pików – wskazuje to na umiarkowaną niestabilność, jednak z ograniczoną liczbą wartości odstających. Natomiast dane typu int cechują się największą zmiennością – linia ma liczne i wysokie piki, co sugeruje, że sortowanie tego typu danych jest najbardziej wrażliwe na ich konkretny układ w pamięci. Ogólnie rzecz biorąc, chociaż wszystkie trzy typy wykazują pewne wahania, algorytm zachowuje względną

przewidywalność i brak ekstremalnych odchyleń czasowych, co potwierdza jego umiarkowaną stabilność działania.



Wykres 22. Stabilność czasów sortowania w serii pomiarów algorytmu Drunk Insertion Sort w zależności od typu danych

Wydajność

Pod względem wydajności, typ danych float okazał się zdecydowanym liderem. Dzięki rozproszonej naturze liczb zmiennoprzecinkowych, algorytm szybciej porządkował dane, nawet mimo losowego pomijania elementów. Typ char osiągnął czasy pośrednie, ponieważ jego wąski zakres wartości prowadził do częstych powtórzeń, co mogło utrudniać rozróżnianie elementów w trakcie sortowania. Natomiast typ int był najwolniejszy, ponieważ szeroki zakres liczb powodował, że nawet niewielkie pominięcia elementów wprowadzały większe trudności w przywracaniu porządku. Przykładowo, algorytm musiał wykonać więcej przesunięć, aby utrzymać poprawny porządek w przypadku danych typu int.

Wnioski

- Typ float jest najbardziej wydajny, ponieważ liczby zmiennoprzecinkowe mają bardziej rozproszoną naturę, co ułatwia algorytmowi porządkowanie danych.
- Typ int miał najgorsze wyniki, ponieważ szeroki zakres liczb powodował trudności w nadrobieniu błędów przy pomijaniu elementów, co skutkowało wydłużeniem czasu sortowania.
- Algorytm pijanego studenta wykazuje stabilność, ale jego czas wykonania jest wrażliwy na typ danych, co może prowadzić do dużych różnic w wydajności.

Podsumowanie

Algorytmy:

- **Quick Sort** zdecydowanie wypadł najlepiej – był najszybszy i świetnie radził sobie niezależnie od tego, jakie dane mu podano. Co ciekawe, losowe dane często dawały mu lepsze wyniki niż uporządkowane – co może być zaskakujące, ale wynika z jego strategii doboru pivota.
- **Heap Sort** był bardziej przewidywalny – nie miał aż takich skoków czasowych jak Quick Sort, ale był zauważalnie wolniejszy. Dobrze sobie radził z różnymi układami danych, ale w praktyce jego moc nie do końca przekłada się na szybkość.
- **Insertion Sort**, mimo że prosty i ciekawy w wersji z wyszukiwaniem binarnym, niestety nie nadążał za resztą przy większych zbiorach. Był raczej dobrym przykładem, jak prosty algorytm może działać dobrze tylko przy małych ilościach danych.
- **Drunk Insertion Sort**, z jednej strony, ten algorytm na pewno nie jest najszybszy i ma swoje momenty, gdy zupełnie nie wie, co robi. Kto wie, może przy niskim stopniu „pijaństwa” i przy odpowiednich danych (np. częściowo posortowanych) taki algorytm mógłby zaskoczyć efektywnością (jak w przypadku zbioru posortowanego malejąco) – ale to już jest bardziej zabawa, niż prawdziwa konkurencja dla klasycznych metod sortowania, wciąż jest to algorytm, który „odstaje” od klasycznych, bardziej deterministycznych metod sortowania.

Program oraz testowanie:

Praca nad programem przebiegała dość sprawnie, choć nie obyło się bez wyzwań. Najwięcej czasu zajęło mi opracowanie i dopracowanie wariantu algorytmu nazwanego „pijanym studentem”. Jego działanie miało charakter losowy i chaotyczny, przez co w niektórych próbach sortowanie mogło trwać niezwykle długo. Mimo to udało się osiągnąć zamierzony efekt – posortowaną tablicę – co dało dużą satysfakcję. Sam pomysł był ciekawy i pozwalał na zastosowanie bardziej kreatywnego podejścia do klasycznego problemu.

Etap testowania okazał się najbardziej czasochłonny. Duża liczba pomiarów i początkowe rozbieżności w wynikach wymagały dokładnej analizy. W wielu przypadkach różnice w czasach działania wzbudzały niepokój, jednak z czasem okazało się, że były to naturalne wahania związane z różnym rozkładem danych wejściowych, a nie błędy w kodzie.

Kluczowym momentem projektu była analiza stabilności czasowej algorytmu Insertion Sort – zarówno przed, jak i po poprawce implementacji. W pierwotnej wersji (Wykres 2.1) algorytm wykazywał nienaturalną stabilność czasów, niezależnie od liczby prób i danych wejściowych. Początkowo wydawało się to zaletą, jednak po głębszej analizie odkryto, że przyczyną była błędna deklaracja zmiennej `j` jako `size_t`. Ponieważ jest to typ bez znaku, pętla mogła kończyć się zbyt wcześnie, co skutkowało niepełnym sortowaniem.

Po poprawieniu tej usterki (zmiana typu `j` na `int`), algorytm zaczął działać poprawnie, co widać na Wykresie 2.2. Czasy sortowania stały się bardziej zróżnicowane i lepiej

odzwierciedlały złożoność $O(n^2)$ – zwłaszcza dla większych danych pojawiły się naturalne fluktuacje i szersze zakresy pomiarów.

Projekt pokazał, jak pozornie drobna zmiana w typie zmiennej może diametralnie wpłynąć na poprawność działania i interpretację wyników. Jednocześnie był to wartościowy proces – zarówno od strony algorytmicznej, jak i implementacyjnej – oraz okazja do sprawdzenia własnych pomysłów w praktyce.